

RÉPUBLIQUE TUNISIENNE

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université de Sfax

**Institut Supérieur d'Informatique et de Multimédia
de Sfax**
(ISIMS)



Cycle ingénieur — Génie Logiciel

2^e année

Rapport de Projet de Fin d'Année
(PFA)

Sujet

**Conception et réalisation d'une application web
de support client assisté par un agent IA**
(Op Support IA)

Réalisé par

Mohamed Chaabene

Organisme d'accueil

SINAPS TEC

*Solutions Innovations For Networks,
Applications & Portable Systems*

Encadrant académique

Non renseigné dans les sources disponibles

Encadrant professionnel

Kais Gharred

sinapstec@gmail.com

Année universitaire 2026–2027

Stage du 22 juin 2026 au 6 septembre 2026

Résumé

Ce rapport présente le travail réalisé dans le cadre du Projet de Fin d'Année (PFA) de 2^e année du cycle ingénieur en Génie Logiciel à l'ISIMS de Sfax. Le stage, effectué à distance au sein de **SINAPS TEC** (Solutions Innovations For Networks, Applications & Portable Systems) du 22 juin au 6 septembre 2026, a consisté à concevoir et développer **Op Support IA**, une solution web et mobile de support client assisté par un agent d'intelligence artificielle.

Le système permet à un client d'ouvrir une conversation, d'obtenir une réponse automatique fondée sur une base de connaissances (approche RAG) et, si besoin, de basculer vers un agent humain en temps réel. Des suggestions d'action rapide accompagnent les réponses IA, tandis que les messages, horodatages, avatars et changements de statut sont persistés. Un espace agent gère la file d'attente des demandes escaladées ; un espace administrateur valide les comptes, consulte l'historique et affiche des indicateurs (volume, résolution IA ou humaine, satisfaction, délai de réponse).

La solution est organisée autour d'un frontend web **Next.js**, d'un client mobile **Expo / React Native** et d'un backend **Node.js / Express**, avec persistance **MongoDB**, authentification JWT et Google, génération de réponses via **Gemini** et un retriever local TF-IDF / similarité cosinus. Les deux clients partagent le backend sans être fonctionnellement identiques : le web propose des actions rapides contextuelles, tandis que le mobile propose des suggestions statiques et nécessite une build native pour Google. Des tests automatisés (Jest, Supertest) couvrent l'API, l'authentification, le téléversement, le retriever, les statuts, les horodatages et l'isolation des données de démonstration.

Mots-clés : support client, agent IA, RAG, Next.js, Express, MongoDB, Socket.io, JWT, PFA.

Dédicace

À mes parents,

À vous qui m'avez soutenu tout au long de ce parcours, par votre patience, vos encouragements et votre confiance. Ce travail est le fruit d'un effort que vous avez rendu possible.

Mohamed Chaabene

Remerciements

Nous tenons tout d’abord à remercier **Kais Gharred**, encadrant professionnel au sein de SINAPS TEC, pour nous avoir proposé le sujet *Op Support IA*, pour les orientations techniques communiquées (stack, périmètre fonctionnel, choix de Gemini et de MongoDB) et pour le suivi à distance tout au long du stage.

Nous remercions également l’ensemble du corps enseignant de l’Institut Supérieur d’Informatique et de Multimédia de Sfax pour les connaissances et les méthodes mobilisées dans ce projet.

Enfin, nous remercions nos parents pour leur soutien constant pendant ces semaines de travail en autonomie.

Table des matières

Résumé	i
Dédicace	ii
Remerciements	iii
Liste des abréviations	ix
Introduction générale	1
1 Présentation du cadre du projet	3
1.1 Présentation de SINAPS TEC	3
1.2 Contexte du projet	4
1.2.1 Cadre académique et déroulement du stage	4
1.2.2 Contexte fonctionnel et projet Op Support IA	5
1.3 Problématique	6
1.4 Objectifs	7
1.5 Méthodologie et démarche	7
1.6 Conclusion	7
2 Analyse et spécification des besoins	8
2.1 Introduction	8
2.2 Identification des acteurs	8
2.3 Besoins fonctionnels	9
2.4 Besoins non fonctionnels	10
2.5 Exigences et périmètre	11
2.6 Contraintes et limites du périmètre	13
2.6.1 Contraintes	13
2.6.2 Hors périmètre (à ce stade)	13
2.7 Conclusion	13

3	Conception du système	14
3.1	Introduction	14
3.2	Architecture générale	14
3.3	Architecture logicielle	15
3.4	Diagramme de cas d'utilisation	16
3.4.1	CU01 — S'identifier (client)	17
3.4.2	CU02 — Envoyer un message	17
3.4.3	CU03 — Escalader	17
3.4.4	CU04 — Traiter une demande (agent)	17
3.4.5	CU05 — Administrer	17
3.5	Modèle de données	17
3.6	Diagramme de classes	20
3.7	Diagrammes de séquence	21
3.7.1	Envoi d'un message client et réponse IA	21
3.7.2	Escalade vers un humain	23
3.7.3	Clôture et satisfaction	23
3.8	Cycle de vie d'une conversation	23
3.9	Authentification et gestion des droits	24
3.10	Conception du RAG	25
3.10.1	Retriever	25
3.10.2	Génération et repli	25
3.10.3	Escalade	25
3.11	Conclusion	26
4	Réalisation	27
4.1	Environnement et choix techniques	28
4.1.1	Déploiement et hébergement	30
4.2	Organisation du code	31
4.3	Backend : API REST, middleware et WebSockets	32
4.3.1	Routes	33
4.3.2	Temps réel	33
4.3.3	Statistiques	34
4.3.4	Téléversement	34
4.4	Moteur IA	34
4.5	Frontend	35
4.5.1	Espace client	35

4.5.2	Espace agent	36
4.5.3	Espace administrateur	37
4.5.4	Client mobile	38
4.6	Données de démonstration	43
4.7	Mise en route (développement local)	43
5	Tests et validation	45
5.1	Introduction	45
5.2	Stratégie de test	45
5.3	Tests fonctionnels	46
5.4	Tests backend automatisés	47
5.4.1	Intégration API	47
5.4.2	Authentification, conversations et messages	48
5.4.3	Upload et retriever RAG	48
5.4.4	Résultat de la suite	49
5.5	Résultats et anomalies corrigées	49
5.6	Validation finale	50
5.7	Conclusion	50
	Conclusion générale	51
A	Annexes	54
A.1	Glossaire des statuts de conversation	54
A.2	Exemple d'entrée de la base de connaissances	54
A.3	Comptes du script de seed (environnement de démo)	55

Table des figures

1.1	Logo de SINAPS TEC, organisme d'accueil du stage	3
3.1	Architecture logique d'Op Support IA (vue déployable en local)	15
3.2	Diagramme global des cas d'utilisation de SINAPS	16
3.3	Modèle de domaine et principales relations persistées de SINAPS	20
3.4	Envoi d'un message client, recherche RAG et génération de la réponse IA .	21
3.5	Escalade explicite vers un agent humain et retour vers l'assistant IA	22
3.6	Cycle de vie d'une conversation selon son statut et son responsable	24
4.1	Principales technologies mobilisées dans la réalisation de SINAPS	28
4.2	Architecture générale et stack technologique de SINAPS	30
4.3	Architecture de déploiement et d'hébergement de SINAPS	31
4.4	Pipeline de recherche augmentée par génération (RAG) de SINAPS	34
4.5	Évaluation de la satisfaction à la clôture d'une conversation	35
4.6	Conversation client avec réponse RAG/Gemini et actions rapides	36
4.7	Escalade d'une conversation vers la file des agents	37
4.8	Tableau de bord administrateur de SINAPS TEC	38
4.9	Écran d'identification directe du client dans l'application mobile SINAPS .	40
4.10	Conversation mobile avec réponse de l'assistant IA fondée sur le RAG et actions rapides	41
4.11	Escalade explicite vers un agent humain : états En attente, Agent demandé et retour Mode IA	42
4.12	Évaluation de la satisfaction et clôture d'une conversation mobile	43

Liste des tableaux

1.1	Fiche signalétique du stage	5
2.1	Acteurs du système	9
2.2	Exigences fonctionnelles et statut dans le dépôt	11
3.1	Collection users (clients)	18
3.2	Collection agents (opérateurs et administrateurs)	18
3.3	Collection conversations	19
3.4	Collection messages	20
3.5	Politique d'authentification	24
4.1	Stack réellement utilisée	28
4.2	Arborescence simplifiée	32
4.3	Principales routes HTTP	33
5.1	Scénarios fonctionnels vérifiés par les tests backend	47
5.2	Exemples de cas d'intégration API	48
A.1	Statuts et modes de prise en charge	54
A.2	Comptes créés par <code>npm run seed</code>	55

Liste des abréviations

Sigle	Signification
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
FAQ	Foire Aux Questions
HTTP	HyperText Transfer Protocol
IA	Intelligence artificielle
ISIMS	Institut Supérieur d'Informatique et de Multimédia de Sfax
JWT	JSON Web Token
LLM	Large Language Model
MVP	Minimum Viable Product
OAuth	Open Authorization
PFA	Projet de Fin d'Année
PFE	Projet de Fin d'Études
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SLA	Service Level Agreement
SPA	Single Page Application
TF-IDF	Term Frequency – Inverse Document Frequency
UML	Unified Modeling Language
UX	User Experience

Introduction générale

Le support client constitue un point de contact critique entre une organisation et ses utilisateurs. Lorsqu'il repose uniquement sur des opérateurs humains, il subit des files d'attente, des délais variables et une répétition des mêmes questions (suivi de commande, remboursement, mot de passe, etc.). À l'inverse, une FAQ statique ne dialogue pas, n'orchestre pas un dossier et n'offre pas de bascule vers un humain lorsque la demande sort du cadre prévu.

Le sujet de stage proposé par **Kais Gharred** au sein de **SINAPS TEC** vise à concevoir une application de support « assistée par un agent IA », proche d'un canal de discussion de type Slack : l'utilisateur pose une question, un agent IA s'appuie sur une **base de connaissances** (approche RAG) pour répondre, un **workflow** suit la demande, et l'utilisateur peut à tout moment **basculer vers un opérateur humain**. Un administrateur valide les comptes des agents et supervise l'historique ainsi que des indicateurs opérationnels de satisfaction et de délai de réponse.

Ce Projet de Fin d'Année s'inscrit dans le cycle ingénieur en **Génie Logiciel** (2^e année) à l'**ISIMS** (Université de Sfax). Le stage s'est déroulé **à distance**, du **22 juin 2026** au **6 septembre 2026**, et a été réalisé **seul**. Le livrable logiciel comprend une application **web** fullstack et un client mobile **Expo / React Native**, tous deux connectés au backend commun. Le web constitue le client principalement documenté et validé ; le mobile couvre le parcours client ainsi que des écrans agent et administrateur, mais ne constitue pas une copie fonctionnellement identique du web et aucune recette formelle de cette version n'est documentée.

Les objectifs poursuivis ont été les suivants :

- analyser le besoin exprimé dans le cahier des charges du stage et le traduire en exigences réalisables à l'échelle d'un PFA ;

- concevoir une architecture frontend / backend / base de données cohérente avec les choix validés (Next.js, Express, MongoDB, Gemini) ;
- implémenter les trois espaces (client, agent, administrateur), le moteur IA et l'escalade temps réel ;
- sécuriser les accès (sessions client, JWT agent/admin, contrôle d'appartenance des conversations) ;
- valider le comportement par des tests automatisés et examiner les intégrations configurées avec Google Sign-In, Gemini et MongoDB.

Le présent rapport est organisé comme suit. Le **chapitre 1** situe le cadre académique et professionnel du projet. Le **chapitre 2** analyse l'existant, le cahier des charges et les besoins. Le **chapitre 3** présente la conception (architecture, UML, données, RAG, authentification). Le **chapitre 4** décrit la réalisation effective. Le **chapitre 5** expose les tests présents dans le dépôt. Une **conclusion générale** dresse le bilan et les perspectives, sans inventer de résultats d'exploitation.

Chapitre 1

Présentation du cadre du projet

1.1 Présentation de SINAPS TEC

L'organisme d'accueil du stage est **SINAPS TEC**, acronyme de *Solutions Innovations For Networks, Applications & Portable Systems*, en Tunisie. Son domaine d'activité, tel qu'il est pertinent pour ce projet, concerne l'informatique, les logiciels, les réseaux et les systèmes.

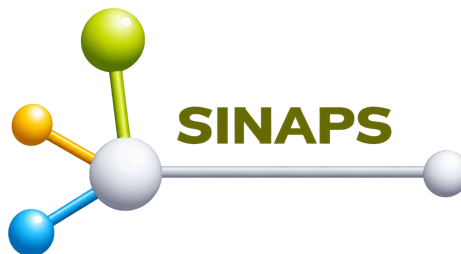


Figure 1.1 – Logo de SINAPS TEC, organisme d'accueil du stage

Le stage a été proposé et suivi par **Kais Gharred**, encadrant professionnel, contacté à l'adresse sinapstec@gmail.com. Kais Gharred est également référencé par l'ANCS comme auditeur en cybersécurité agréé rattaché à **RESYS-CONSULTANTS**. Cette référence professionnelle ne signifie pas que SINAPS TEC et RESYS-CONSULTANTS sont la même entité : il s'agit de deux organisations distinctes, et aucune relation de filiale ou de succursale n'est établie dans les sources disponibles.

Le travail a été réalisé **à distance**, sans présence physique dans des locaux décrits dans

les documents disponibles. Le présent rapport se limite donc aux informations vérifiables dans le cadre du projet et ne déduit ni coordonnées officielles, ni forme juridique, ni effectif, ni historique institutionnel.

Dans le cadre de ce PFA, SINAPS TEC intervient comme **entreprise d’accueil et commanditaire du sujet** : le besoin fonctionnel, la stack suggérée et le périmètre (chat de support, RAG, escalade humaine, administration) ont été définis par l’encadrant professionnel, puis réalisés en autonomie.

1.2 Contexte du projet

1.2.1 Cadre académique et déroulement du stage

L’**Institut Supérieur d’Informatique et de Multimédia de Sfax** (ISIMS) est un établissement de l’**Université de Sfax**. Il forme notamment des ingénieurs en informatique. Le présent travail correspond à un **Projet de Fin d’Année** (PFA) de **2^e année** du **cycle ingénieur en Génie Logiciel**, année universitaire **2026–2027**.

Le PFA précède le **Projet de Fin d’Études** (PFE). Il vise à confronter l’étudiant à un développement logiciel complet — spécification, conception, implémentation, tests — dans un volume compatible avec un stage de quelques mois, sans prétendre à l’ampleur d’un PFE de fin de cycle.

L’identité et le grade de l’encadrant académique ne figurent pas dans les sources disponibles pour ce rapport.

Table 1.1 – Fiche signalétique du stage

Intitulé	Conception et réalisation d’une solution web et mobile de support client assisté par un agent IA (Op Support IA)
Organisme	SINAPS TEC
Encadrant professionnel	Kais Gharred (sinapstec@gmail.com)
Encadrant académique	Non renseigné dans les sources disponibles
Étudiant	Mohamed Chaabene
Période	du 22 juin 2026 au 6 septembre 2026
Durée	environ deux mois et demi
Modalité	à distance
Équipe projet	un seul stagiaire (réalisation intégrale)

Le document de sujet fourni par l’encadrant mentionnait une durée de référence de **12 semaines** et un calendrier « juillet 2026 ». La période retenue pour ce rapport est celle effectivement indiquée pour le stage : **22 juin – 6 septembre 2026**. Le même document évoquait une « équipe agile » ; dans les faits, le développement a été **individuel** et **informel**.

1.2.2 Contexte fonctionnel et projet Op Support IA

Une demande de support typique (suivi de commande, retard, remboursement, mot de passe, code promotionnel) traverse plusieurs étapes : qualification, recherche d’information, réponse, éventuellement transfert vers un humain, puis clôture et recueil d’avis. Sans outil dédié, ces étapes se perdent dans la messagerie personnelle ou un standard téléphonique. Les conséquences habituelles sont :

- une charge répétitive pour les opérateurs sur des questions déjà documentées ;
- l’absence de **traçabilité** (statut, interlocuteur, pièces jointes, historique) ;
- l’impossibilité de distinguer ce qui a été résolu par une FAQ de ce qui a nécessité un humain ;
- un faible pilotage (délai de première réponse, satisfaction).

Le projet, intitulé dans le cahier des charges *Projet de stage — Support by IA / Op Support IA*, consiste à concevoir une application de support dans laquelle :

1. le client émet une demande dans une interface de chat ;
2. un **agent IA** interroge une base de connaissances (RAG) et répond ;
3. un **workflow** suit la demande (prise en charge IA, attente d'un humain, résolution) ;
4. le client peut **basculer** vers un opérateur humain ;
5. l'échange peut inclure des pièces jointes ;
6. la clôture s'accompagne d'une **note de satisfaction** ;
7. l'administrateur valide les agents et consulte statistiques et historique.

Le produit livré comprend une application **web** (interface responsive) et un client mobile **Expo / React Native** connecté au même backend. Le mobile couvre l'authentification, le chat, les réponses IA, l'escalade humaine, la clôture avec satisfaction, les pièces jointes, le temps réel Socket.io, ainsi que des écrans agent et administrateur. Il ne constitue toutefois pas une copie fonctionnellement identique du web : l'interface mobile propose des actions rapides contextuelles attachées aux réponses IA éligibles, avec une présentation adaptée aux petits écrans et différente de celle du web. La connexion Google mobile nécessite une build native Expo configurée. Aucune recette formelle de cette version n'est documentée.

1.3 Problématique

Une FAQ seule suffit pour les cas les plus fréquents mais :

- elle n'entretient pas un fil de discussion ;
- elle ne déclenche pas d'escalade ni d'assignation ;
- elle n'enregistre ni SLA ni note de satisfaction.

À l'inverse, un chat 100 % humain (ou un e-mail) offre la souplesse d'un opérateur, mais chaque message consomme du temps humain, y compris pour les demandes déjà couvertes par la connaissance métier. Le sujet cherche donc un **compromis** : l'IA en première ligne, l'humain sur demande ou lorsque la base ne suffit pas.

Le sujet du stage pose ainsi la question d'ingénierie logicielle suivante : **comment automatiser une partie du support tout en conservant une bascule humaine fluide et une supervision administrative ?**

1.4 Objectifs

À partir du cahier des charges et des choix techniques retenus avec l'encadrant (React / Next.js, Express, MongoDB, Gemini, JWT), les objectifs opérationnels ont été :

- disposer d'un chat client francophone, authentifié (Google ou identification par nom et e-mail) ;
- répondre automatiquement via RAG + Gemini, avec un repli local si l'API générative est indisponible ;
- permettre l'escalade et le retour vers l'IA, l'affectation d'un agent, et la clôture notée ;
- offrir un espace agent (file, réponses, pièces jointes) et un espace admin (validation, SLA, historique) ;
- persister les données sur MongoDB et documenter l'utilisation de MongoDB Atlas pour l'environnement distant, puis diffuser les mises à jour par WebSocket (Socket.io) ;
- documenter et tester le backend (santé API, autorisations, upload, retrieve).

1.5 Méthodologie et démarche

Le stage n'a pas suivi un cadre Scrum (pas de sprints planifiés, pas de daily, pas de tableau Kanban documenté). Le travail s'est organisé de manière **itérative et informelle** : lecture du sujet, choix d'architecture, implémentation fullstack, sécurisation des routes, ajout des tests, puis rédaction du présent rapport.

Les outils utilisés en plus du code source sont : **Visual Studio Code**, **Cursor**, **Anti-gravity**, **GitHub**, ainsi que les consoles **Google** (OAuth, Gemini) et **MongoDB Atlas**. Postman est mentionné dans la documentation du projet pour les vérifications manuelles de l'API ; aucun compte rendu détaillé ni collection Postman n'est conservé dans le dépôt.

1.6 Conclusion

Ce chapitre a présenté le cadre académique et professionnel du PFA, le contexte fonctionnel d'Op Support IA, la problématique du support client ainsi que les objectifs et la démarche retenue. Le chapitre suivant précise les acteurs, les besoins, les exigences et les contraintes qui ont guidé la conception et la réalisation de la solution.

Chapitre 2

Analyse et spécification des besoins

2.1 Introduction

Ce chapitre transforme la problématique présentée au chapitre 1 en besoins et exigences pour le système Op Support IA. Le cahier des charges ne décrivait pas un logiciel interne déjà en production chez SINAPS TEC, mais un **produit à construire** : canal de chat, agent IA, workflow, multi-agents, contenus riches, clôture et satisfaction, ainsi que tableau de bord. L'existant fonctionnel au début du stage était donc **nul** : aucune base de code héritée n'a été fournie et l'application a été écrite intégralement pendant le PFA.

Les outils du marché (chats de type Slack, widgets de support et assistants RAG) constituent uniquement des **repères d'expérience utilisateur** pour le chat « Slack light » demandé dans le sujet ; ils ne représentent pas un système existant de SINAPS TEC. L'analyse ci-dessous distingue les besoins du système, les exigences retenues et leur statut dans le dépôt actuel.

2.2 Identification des acteurs

Quatre rôles ou composants interviennent dans le fonctionnement du système. Le tableau 2.1 présente les acteurs humains ; le diagramme de cas d'utilisation du chapitre 3 reprend les acteurs humains.

Table 2.1 – Acteurs du système

Acteur	Rôle
Client	Utilisateur final du support. S'identifie par Google ou par nom et e-mail. Consulte et alimente sa conversation.
Agent de support	Compte créé par inscription, validé par l'admin. Traite les fils escaladés.
Administrateur	Même modèle de compte qu'un agent, avec le rôle admin . Valide ou refuse les agents, consulte stats et historique.

Le système IA n'est **pas** un acteur UML au sens d'un utilisateur humain : c'est un **composant interne** qui interroge la base de connaissances, peut appeler Gemini et produit des messages de type **sender = ia**. Le backend, MongoDB et Socket.io sont également des composants techniques internes, et non des acteurs humains supplémentaires.

2.3 Besoins fonctionnels

La solution retenue est une application web exposant trois espaces, complétée par un client mobile connecté au même backend :

1. **Client** (/) : identification, conversation unique active, réponses IA, pièces jointes, escalade / retour IA, clôture et satisfaction ;
2. **Agent** (/agent) : connexion e-mail / mot de passe, file des conversations, réponses humaines, clôture ;
3. **Administrateur** (/admin) : validation des inscriptions agents, statistiques, historique filtrable.

Les besoins fonctionnels couvrent donc l'authentification des utilisateurs, la création et la consultation des conversations, l'envoi et la réception des messages, les réponses IA fondées sur la base de connaissances, l'escalade vers un agent humain et le retour vers l'IA. Ils comprennent également la gestion des fils par les agents, les pièces jointes, la clôture avec satisfaction, la validation des comptes agents, l'historique, les filtres et les statistiques du tableau de bord.

Le client mobile Expo / React Native partage les routes HTTP et le serveur Socket.io du backend. Il couvre l'authentification, le chat, les réponses IA, l'escalade humaine, la

clôture avec satisfaction, les pièces jointes, le temps réel, ainsi que des écrans agent et administrateur. Les différences avec le web et le statut partiel de cette réalisation sont précisés dans le tableau des exigences.

Côté serveur, une API REST Express persiste les entités dans MongoDB, appelle un service RAG + Gemini et notifie les clients via Socket.io. La conception de cette architecture est détaillée au chapitre 3 ; la présente section se limite à l'expression des besoins.

2.4 Besoins non fonctionnels

RNF01 — Utilisabilité

Interface en français, simple, adaptée à un usage type Slack light (exigence de design du sujet).

RNF02 — Adaptabilité d'écran

Fonctionnement sur desktop, tablette et smartphone via une UI responsive (Next.js) ; un client mobile Expo / React Native distinct est également présent.

RNF03 — Sécurité

Mots de passe agents hachés (bcrypt) ; JWT ; vérification du jeton Google ; restriction d'accès aux conversations (propriétaire client ou agent/admin) ; messages `ia` non injectables depuis l'extérieur ; upload authentifié et filtré.

RNF04 — Disponibilité du bot

Si `GEMINI_API_KEY` est absente ou l'appel échoue, un repli sur le meilleur extrait RAG (ou un message d'invitation à l'escalade) évite un silence total.

RNF05 — Temps réel

Diffusion des messages et des mises à jour de conversation par Socket.io (salles par conversation et événements globaux).

RNF06 — Testabilité

Suite Jest / Supertest / MongoMemoryServer sur le backend.

RNF07 — Portabilité de configuration

Secrets et URI via variables d'environnement ; exemples `.env.example`.

Les besoins non fonctionnels décrits ici restent limités aux propriétés effectivement présentes dans le sujet ou dans le dépôt ; aucune garantie de performance, de disponibilité en production ou de confidentialité supplémentaire n'est avancée sans résultat vérifié.

2.5 Exigences et périmètre

Les exigences ci-dessous sont dérivées du document de sujet *Projet Stage – Op Support IA* et de ce qui a été effectivement spécifié pour le MVP. Le tableau 2.2 indique le statut **dans le dépôt actuel**, afin de ne pas confondre le souhait initial et le réalisé.

Table 2.2 – Exigences fonctionnelles et statut dans le dépôt

Id	Exigence	Statut
RF01	Le client identifie une demande dans une interface de chat.	Réalisé
RF02	Authentification client par compte Google (OAuth).	Réalisé sur le Web et côté serveur (jeton Google vérifié). Une saisie nom/e-mail existe en parallèle ; Google Sign-In natif mobile n'a pas été vérifié.
RF03	L'agent IA répond à partir d'une base de connaissances (RAG).	Réalisé (retrieve TF-IDF + Gemini, repli local).
RF04	Workflow de statuts (en cours, en attente, résolu) et mode IA / humain.	Réalisé
RF05	Bascule vers un opérateur humain à l'initiative du client.	Réalisé
RF06	Retour vers l'IA (désescalade).	Réalisé
RF07	Plusieurs agents (comptes distincts, compétences saisies à l'inscription).	Comptes : réalisé. Routage automatique par compétence : non réalisé .
RF08	Chat avec une UX de type messagerie (bulles, emojis, interface soignée).	Réalisé (web), avec indicateur de saisie, verrouillage pendant la réponse IA et retours d'action.

Suite à la page suivante

Table 2.2 – suite

Id	Exigence	Statut
RF09	Échange de documents, images, vidéos.	Partiellement réalisé : upload et échange de fichiers pris en charge par le backend et le Web ; images et documents disponibles sur mobile. Le flux vidéo mobile n'a pas été implémenté ni vérifié, et l'aperçu « unfurl » de liens n'est pas documenté comme fonctionnalité distincte.
RF10	Clôture et enquête de satisfaction (note 1–5, commentaire).	Réalisé
RF11	Inscription agent, validation ou rejet par l'admin.	Réalisé
RF12	Historique des conversations, recherche et filtre par statut.	Réalisé (espace admin).
RF13	Statistiques : volume, résolution IA vs humain, satisfaction moyenne, temps de réponse.	Réalisé (agrégations MongoDB, pas de chiffres marketing fictifs).
RF14	Application mobile native (React Native / Flutter).	Partiellement réalisé : client Expo / React Native présent et connecté au backend ; aucune version Flutter ni recette formelle documentée.
RF15	File d'attente / messagerie temps réel.	Réalisé (Socket.io).
RF16	Suggestions d'actions rapides après une réponse IA.	Réalisé (confirmation, nouvelle question, escalade ; actions contextualisées).
RF17	Affichage cohérent des messages et des profils.	Réalisé (horodatages persistés, avatars client de repli déterministes, avatars agent).

Les chiffres illustratifs du document de sujet (satisfaction 94 %, 2,4 min, 68 % de résolutions IA, note 4,7/5) sont des **exemples de communication produit**. Ils ne sont pas

des résultats mesurés pendant le stage et ne seront pas présentés comme tels.

2.6 Contraintes et limites du périmètre

2.6.1 Contraintes

- Durée limitée (environ deux mois et demi) et travail **individuel**.
- Stack imposée / validée : web React (réalisé avec Next.js), backend Node/Express, MongoDB, Gemini, JWT — et non Vue, Angular, OpenAI par défaut, ni Kafka.
- Hébergement applicatif : **local** au moment de la rédaction ; base distante documentée avec **MongoDB Atlas**.
- Base de connaissances : jeu d'**exemples** (commandes, remboursements, mot de passe, codes promo), pas un corpus métier SINAPS TEC fourni par ailleurs.

2.6.2 Hors périmètre (à ce stade)

- application iOS / Android native ;
- file de messages (Kafka, RabbitMQ) ;
- RAG vectoriel (embeddings, base vectorielle, LangChain) ;
- affectation automatique d'un agent selon ses compétences ;
- configuration complète du déploiement public non versionnée dans le dépôt : le Web public et l'API Render ont toutefois été vérifiés fonctionnellement par une exécution externe ; les paramètres Vercel/Render et les secrets ne sont pas reproductibles depuis le dépôt seul.

Ces éléments peuvent figurer parmi les perspectives (conclusion), sans être présentés comme livrés.

2.7 Conclusion

Ce chapitre a identifié les acteurs du système, traduit la problématique en besoins fonctionnels et non fonctionnels, puis distingué les exigences réalisées des éléments restant hors périmètre. Le chapitre suivant présente la conception de l'architecture, des données et des principaux comportements de la solution.

Chapitre 3

Conception du système

3.1 Introduction

Ce chapitre présente les choix de conception retenus pour transformer les besoins du chapitre 2 en une architecture, des modèles de données et des scénarios de fonctionnement cohérents. Les diagrammes décrivent le **système implémenté** et distinguent les responsabilités des clients, du backend, des services internes et des acteurs humains.

3.2 Architecture générale

Le système est une architecture composée de deux clients (web et mobile) et d'un backend commun, communiquant par HTTP(S) et WebSocket, avec une base documentaire MongoDB. Le client web repose sur Next.js / React et le client mobile sur Expo / React Native. Le backend expose une API REST/JSON, coordonne l'authentification, les conversations, le moteur RAG/Gemini et les échanges temps réel avec Socket.io.

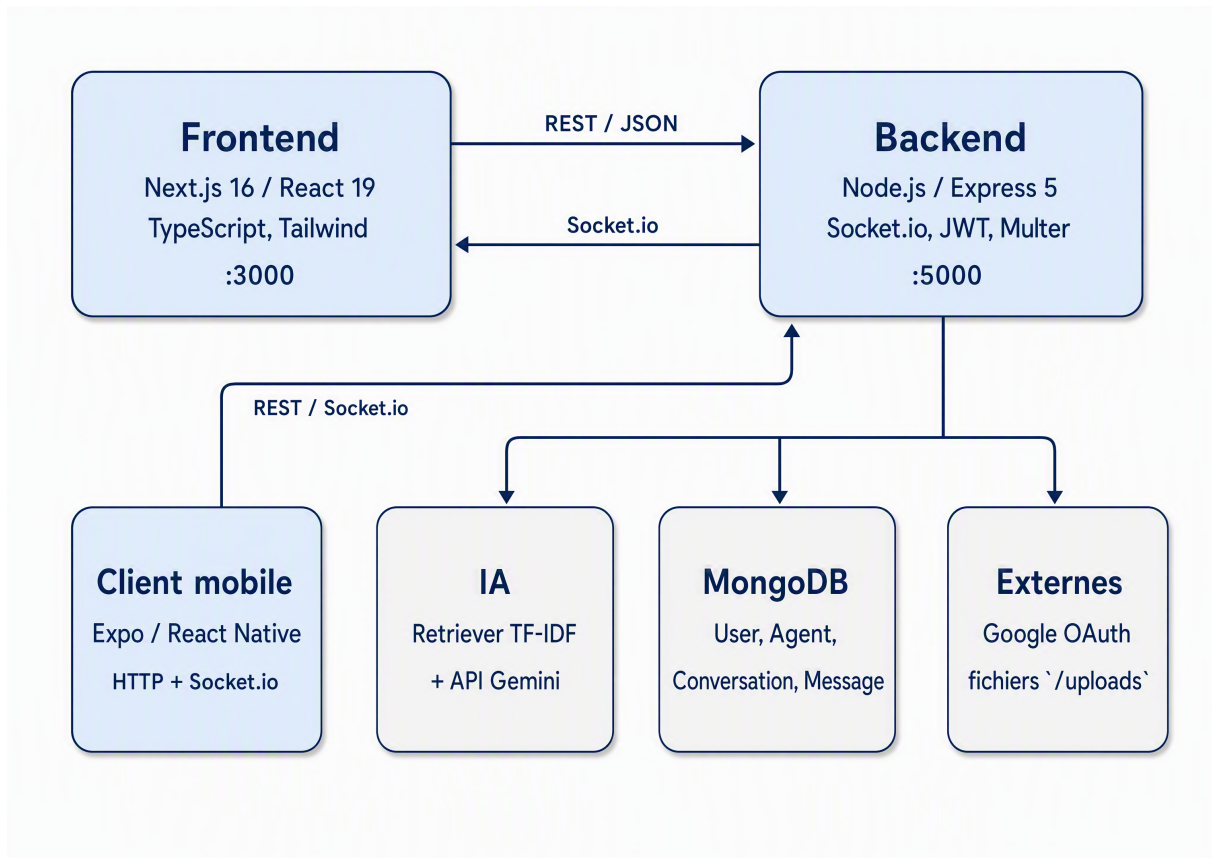


Figure 3.1 – Architecture logique d’Op Support IA (vue déployable en local)

Les flux principaux sont :

1. le navigateur charge l’interface Next.js ;
2. les appels métier passent par l’API Express ;
3. après un message client, si la conversation est en mode IA, le backend appelle le retriever puis Gemini (ou le repli) et enregistre un message **ia** ;
4. Socket.io notifie la salle de conversation et, pour les listes, un événement global ;
5. les fichiers sont stockés sur disque sous **uploads/** et servis en statique ;
6. le client peut demander une prise en charge humaine ; l’agent traite alors la conversation depuis sa file.

3.3 Architecture logicielle

L’organisation logique repose sur quatre ensembles complémentaires :

- les **clients**, web et mobile, qui présentent les interfaces de conversation, d’agent et d’administration et communiquent avec le backend ;
- le **backend**, qui expose l’API REST, applique les contrôles d’accès, persiste les données et coordonne les changements d’état ;
- les **services internes**, qui regroupent l’authentification, le retriever RAG, la génération Gemini, le téléversement et les notifications Socket.io ;
- la **persistance et les services externes**, constitués de MongoDB pour les entités métier, de Google OAuth pour l’identification client et de Gemini pour la génération lorsqu’une clé valide est disponible.

Cette séparation permet aux deux clients de partager les mêmes conversations, messages et règles d'autorisation sans dupliquer la logique métier. Les composants techniques restent distincts des acteurs humains identifiés au chapitre 2.

3.4 Diagramme de cas d'utilisation

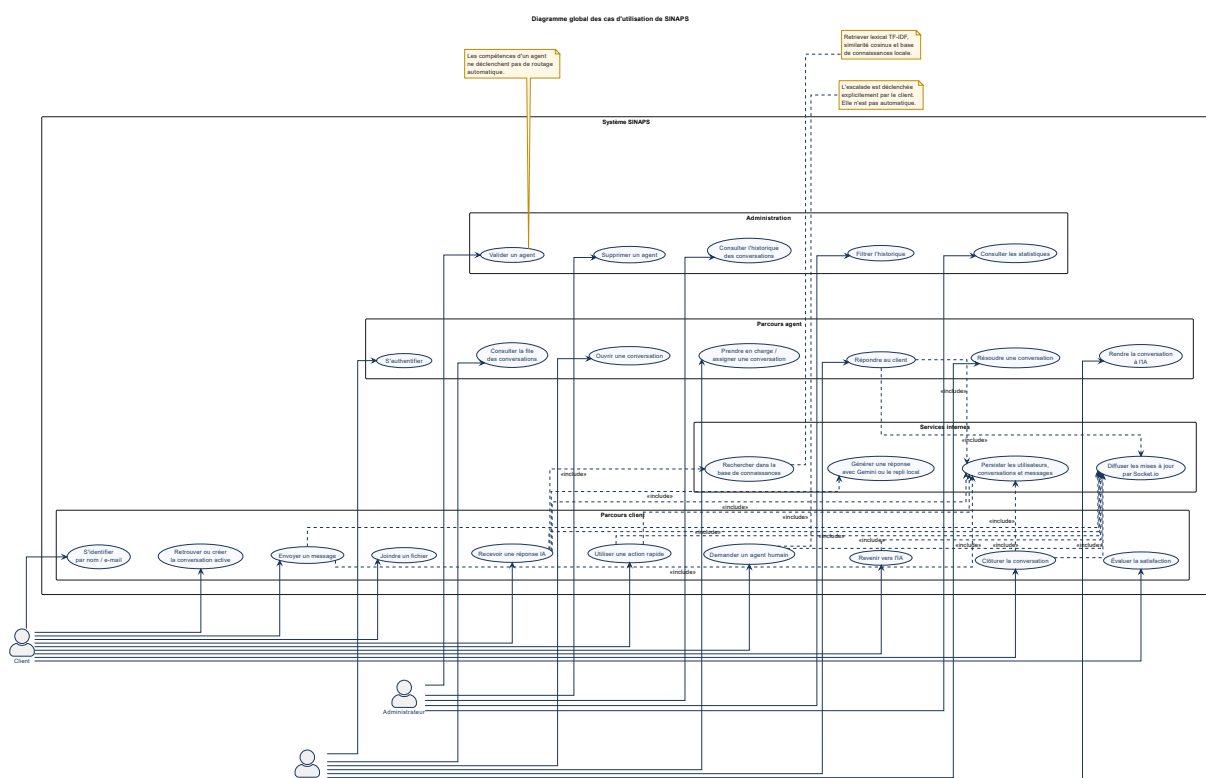


Figure 3.2 – Diagramme global des cas d'utilisation de SINAPS

3.4.1 CU01 — S'identifier (client)

Le client fournit un **jeton Google**, vérifié avec `GOOGLE_CLIENT_ID`, ou un couple nom / e-mail. Le serveur trouve ou crée le profil client et délivre un JWT de rôle `client`. Une conversation non résolue est réutilisée, sinon une nouvelle conversation est créée avec `handledBy = ia` et `status = en_cours`.

3.4.2 CU02 — Envoyer un message

Avec une session client et une conversation ouverte, le message est persisté. Si `handledBy = ia`, le service IA produit une réponse enregistrée comme message `ia`. Les deux messages sont émis sur Socket.io.

3.4.3 CU03 — Escalader

Le client, ou un profil autorisé sur la conversation, passe `handledBy` à `humain` et `status` à `en_attente`. Les agents voient alors le fil dans la file.

3.4.4 CU04 — Traiter une demande (agent)

L'agent authentifié et approuvé consulte la liste, peut être assigné, répond avec `sender = humain`, puis clôture la conversation.

3.4.5 CU05 — Administrer

L'administrateur consulte `GET /api/stats` et `GET /api/conversations`, approuve ou supprime un agent en attente et filtre les conversations.

3.5 Modèle de données

MongoDB stocke quatre collections Mongoose. Les relations sont des références `ObjectId`.

Table 3.1 – Collection **users** (clients)

Champ	Type	Commentaire
googleId	String, unique, requis	Identifiant Google ou, en flux direct, souvent l'e-mail
name, email	String, requis	Profil affiché
avatar	String	URL d'avatar
timestamps	Date	createdAt , updatedAt

Table 3.2 – Collection **agents** (opérateurs et administrateurs)

Champ	Type	Commentaire
name, email	String (e-mail unique)	Identité de connexion
password	String	Hash bcrypt
role	agent admin	Défaut : agent
skills	[String]	Saisies à l'inscription ; non utilisées pour un routage automatique
status	pending approved	Un agent non approuvé ne peut pas se connecter
avatar	String	Optionnel

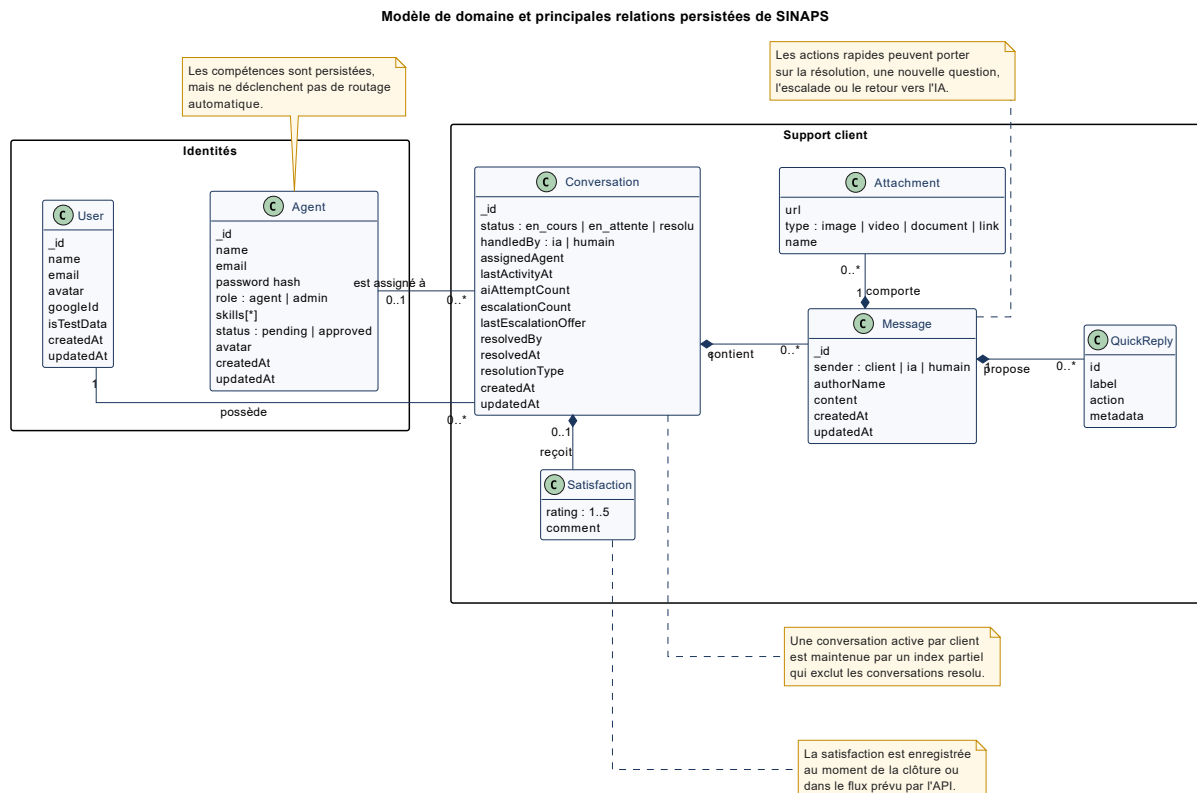
Table 3.3 – Collection conversations

Champ	Type	Commentaire
client	ObjectId → User	Propriétaire
assignedAgent	ObjectId → Agent	Peut être nul
status	en_cours en_attente resolu	Cycle de vie
handledBy	ia humain	Qui répond
satisfaction.rating	Number 1–5	Optionnel à la clôture
satisfaction.comment	String	Optionnel
resolvedBy, resolvedAt	String, Date	Origine et instant de résolution
resolutionType	Enum	Résolution confirmée par le client, par l'agent ou abandon
lastActivityAt	Date	Sélection de la conversation active la plus récente
aiAttemptCount, escalationCount	Number	Suivi des tentatives IA et des escalades
isTestData	Boolean	Marqueur exclu des indicateurs opérationnels

Table 3.4 – Collection messages

Champ	Type	Commentaire
conversation	ObjectId	Fil parent
sender	client ia humain	Provenance
authorName	String	Affichage
content	String	Texte (peut être vide si pièce jointe)
attachments[]	url, type, name	image video document link
quickReplies[]	id, label, action, metadata	Actions proposées après une réponse IA
timestamps	Date	createdAt, updatedAt, utilisés pour l’affichage du fil

3.6 Diagramme de classes

**Figure 3.3** – Modèle de domaine et principales relations persistées de SINAPS

3.7 Diagrammes de séquence

3.7.1 Envoi d'un message client et réponse IA

Le client transmet le message au backend, qui le persiste, interroge le retriever et le service Gemini lorsque la conversation est prise en charge par l'IA. La réponse éventuelle est enregistrée puis diffusée aux clients par Socket.io.

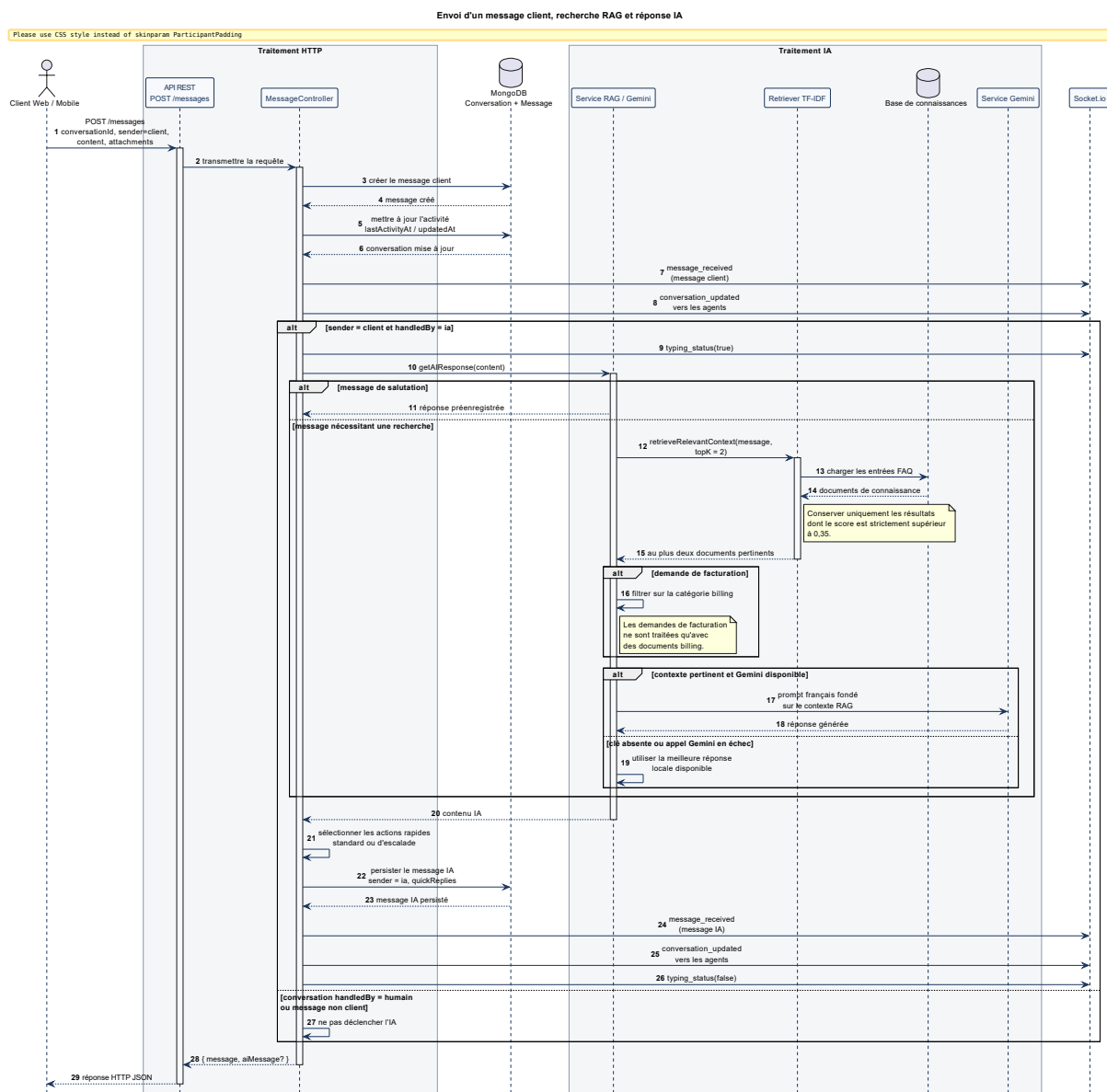


Figure 3.4 – Envoi d'un message client, recherche RAG et génération de la réponse IA

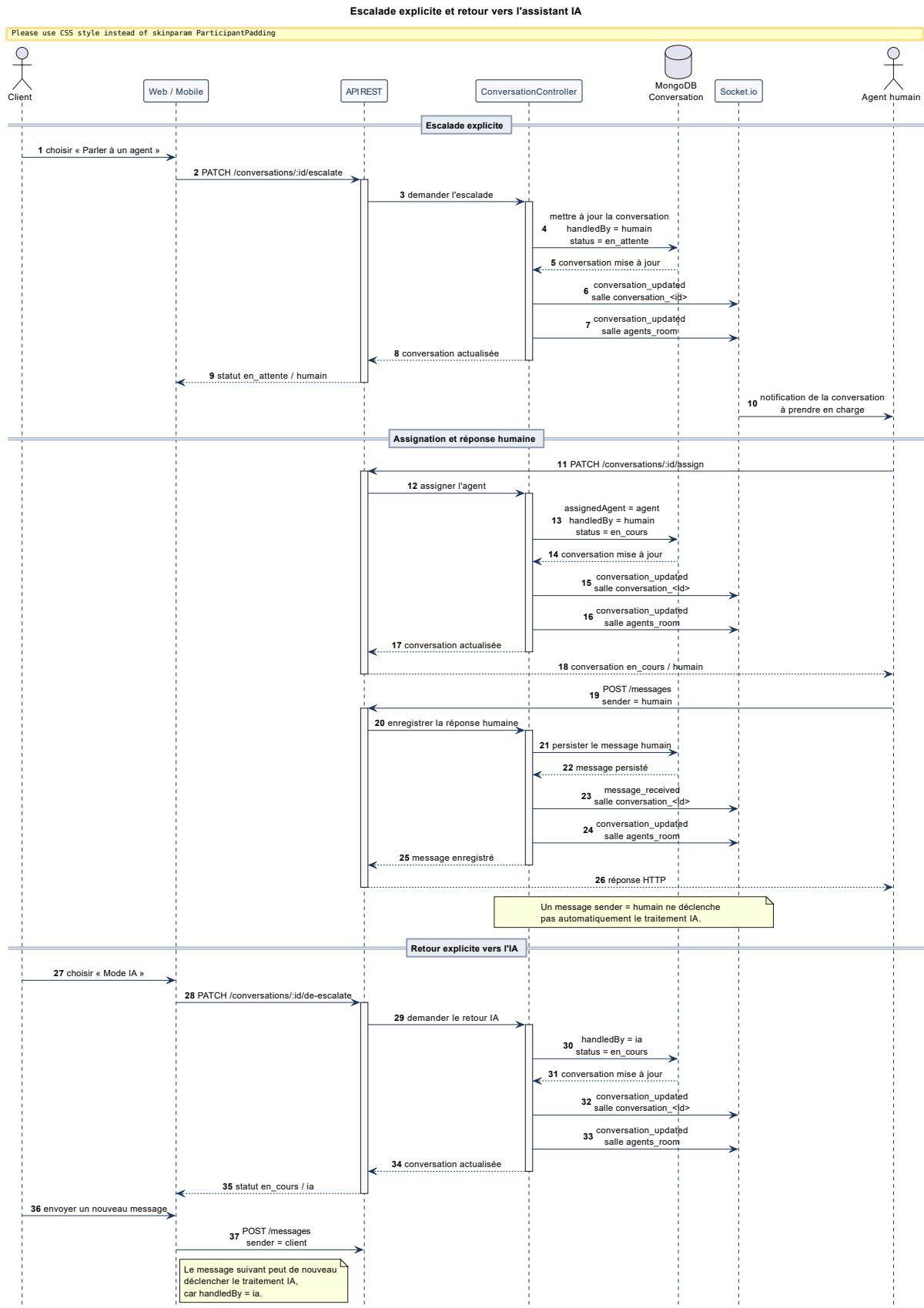


Figure 3.5 – Escalade explicite vers un agent humain et retour vers l'assistant IA

3.7.2 Escalade vers un humain

La figure 3.5 détaille le scénario déclenché explicitement par le client : la conversation est transférée à un agent, puis peut revenir au traitement IA avant sa clôture. Le client appelle `PATCH /conversations/:id/escalate`. Le document passe à `handledBy = humain` et `status = en_attente`. Socket.io émet `conversation_updated` vers la salle ; un événement global permet à l'espace agent de rafraîchir la file.

3.7.3 Clôture et satisfaction

`PATCH /conversations/:id/close` accepte une note entière entre 1 et 5. Une valeur hors intervalle, par exemple 0, n'est pas enregistrée comme satisfaction. Le statut devient `resolu` et la cause de résolution est tracée dans `resolutionType`. Les actions rapides permettent notamment de confirmer une résolution IA, de poser une nouvelle question ou de demander un agent.

3.8 Cycle de vie d'une conversation

Une conversation est créée ou réutilisée lors de l'identification du client. Elle commence avec `handledBy = ia` et `status = en_cours`. Les messages sont alors traités par l'IA lorsque le client écrit dans une conversation ouverte. La figure 3.6 synthétise les états et transitions décrits ci-dessous.

Lorsque le client demande une prise en charge humaine, ou lorsqu'un acteur autorisé effectue cette action, la conversation passe à `handledBy = humain` et `status = en_attente`. Un agent approuvé peut la prendre en charge ; elle est alors éventuellement associée à `assignedAgent` et repasse à `status = en_cours`. L'agent peut répondre, puis clôturer la conversation.

La clôture fait passer le statut à `resolu`. Une note entière de 1 à 5 et un commentaire peuvent être enregistrés ; la résolution conserve également son origine et son instant. Le retour vers l'IA est possible avant la clôture par désescalade. Les statuts et modes associés sont récapitulés dans l'annexe A.1.

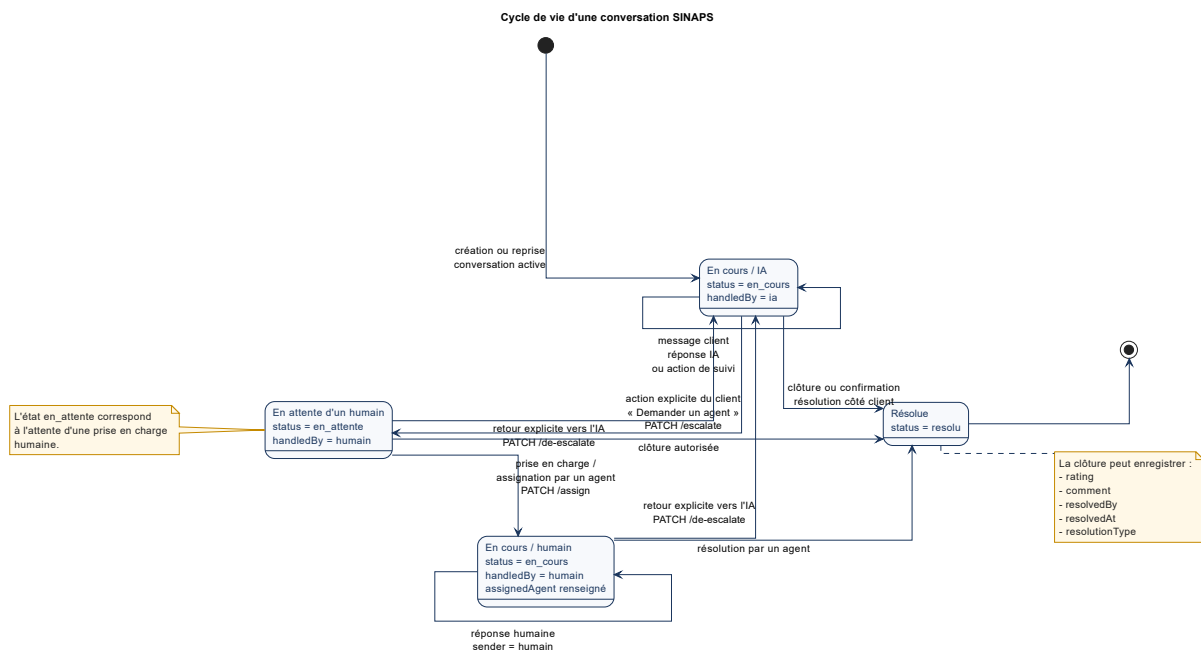


Figure 3.6 – Cycle de vie d’une conversation selon son statut et son responsable

3.9 Authentification et gestion des droits

Deux familles de jetons JWT signés avec `JWT_SECRET` coexistent.

Table 3.5 – Politique d’authentification

Profil	Mécanisme
Client	JWT role=client, 30 jours, après Google (ID token vérifié) ou identification e-mail. Stockage navigateur : <code>sinaps_client</code> .
Agent / admin	JWT role=agent admin, 7 jours, après e-mail + mot de passe. Refus si agent pending. Stockage : <code>sinaps_token</code> .
Middleware	<code>requireAuth</code> , <code>requireAdmin</code> , <code>requireAgentOrAdmin</code> , <code>requireClientAuth</code> , <code>requireConversationAccess</code> , <code>requireSenderAuth</code> , <code>requireAnySession</code> .

Les règles de conception sont les suivantes :

- un client ne lit et ne modifie que **sa** conversation ;
- **sender** = **humain** exige un JWT agent/admin ;
- **sender** = **ia** est refusé s’il vient d’une requête externe ;

— l’upload exige une session client ou agent.

En production, le serveur refuse de démarrer sans `JWT_SECRET`. La mise en uvre détaillée des routes et des middleware est présentée au chapitre 4.

3.10 Conception du RAG

3.10.1 Retriever

La base `data/knowledgeBase.js` contient des paires `{ question, answer }`. Chaque document est normalisé et tokenisé, puis indexé en TF et pondéré par IDF. La requête et les documents sont comparés par **similarité cosinus**. Les extraits dont le score est strictement supérieur à 0,35 sont retenus, avec un maximum de deux documents (`topK = 2`).

Ce n’est **pas** un RAG vectoriel : aucun embedding ni base vectorielle n’est utilisé. Il s’agit d’un RAG « lexical » adapté à une petite FAQ de démonstration.

3.10.2 Génération et repli

Si une clé Gemini valide est présente, les extraits pertinents sont injectés dans un prompt français afin de produire une réponse courte et fondée sur la base de connaissances. Les modèles Gemini configurés sont essayés en cascade. Si la clé est absente, si l’appel échoue ou si aucun document pertinent n’est disponible, le système renvoie la meilleure réponse locale lorsqu’elle existe, sinon un message invitant à demander l’aide d’un agent humain.

3.10.3 Escalade

L’escalade n’est **pas automatique** selon un score de confiance. Elle est **décidée par l’utilisateur** ou par un acteur autorisé sur la conversation via l’action prévue à cet effet. Le mécanisme RAG peut conduire à proposer cette aide humaine lorsqu’aucune réponse pertinente n’est disponible, mais le changement de prise en charge reste une action explicite.

3.11 Conclusion

Ce chapitre a présenté l'architecture générale et logicielle, les cas d'utilisation, le modèle de données, les séquences, le cycle de vie des conversations, l'authentification et la conception du RAG. Le chapitre suivant décrit la réalisation concrète de ces choix dans les applications web, mobile et backend.

Chapitre 4

Réalisation

Ce chapitre décrit l'implémentation **effective** : organisation du code, choix techniques, points d'implémentation notables et écrans illustrés par les captures intégrées.

4.1 Environnement et choix techniques

Table 4.1 – Stack réellement utilisée

Catégorie	Choix réellement utilisés
Frontend Web	Next.js 16 (App Router), React 19 et TypeScript
Backend / API	Node.js et Express 5, avec une API REST/JSON
Application mobile	React Native 0.86, Expo 57 et TypeScript
Données	MongoDB avec Mongoose 9 ; MongoDB Atlas pour l'environnement distant documenté
IA / RAG	Google Gemini et retriever local fondé sur TF-IDF et la similarité cosinus
Communication temps réel	Socket.io côté backend, Web et mobile
Authentification	JWT pour les sessions et Google OAuth pour l'identification client
Tests	Jest, Supertest et MongoDB Memory Server ; Postman est mentionné dans la documentation pour les vérifications manuelles de l'API
Versionnement / déploiement	Git et GitHub ; Vercel pour le Web et Render pour le backend

La figure 4.1 présente visuellement les principales technologies mobilisées dans cette stack.

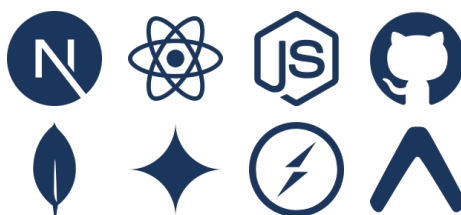


Figure 4.1 – Principales technologies mobilisées dans la réalisation de SINAPS

Lecture de la stack. Le terme **MERN** désigne ici l'association MongoDB, Express.js, React et Node.js. Next.js est le framework utilisé autour de l'application React web ; TypeScript apporte un typage statique aux clients web et mobile. Express.js expose l'API

REST du backend Node.js, avec des échanges JSON, tandis que MongoDB assure la persistance des utilisateurs, agents, conversations et messages. Mongoose définit les schémas utilisés par le backend.

Client mobile. React Native et Expo constituent une application séparée du frontend Web, partageant le backend REST et Socket.io. Cette stack mobile n'est donc pas une composante supplémentaire de MERN.

IA et recherche documentaire. Le service Google Gemini génère les réponses lorsqu'il est disponible. Avant cette génération, le retriever local normalise les textes, calcule des scores TF-IDF et compare les vecteurs par similarité cosinus afin de sélectionner les éléments pertinents de la base de connaissances. En l'absence de réponse Gemini exploitable, le backend dispose également d'un repli local.

Temps réel et authentification. Socket.io complète l'API REST pour diffuser les messages et les mises à jour de conversations. Google OAuth intervient dans l'identification client, tandis que des jetons JWT représentent les sessions client, agent et administrateur et sont contrôlés par les middleware du backend.

Tests et variantes MongoDB. Jest orchestre les tests, Supertest vérifie les routes HTTP et MongoDB Memory Server fournit une base isolée pour les tests automatisés. **MongoDB** désigne la technologie de persistance ; **MongoDB Atlas** désigne le service distant documenté pour l'environnement de production ; **MongoDB Memory Server** est réservé à l'isolation des tests. Git et GitHub assurent le versionnement et l'hébergement du dépôt. Postman est mentionné dans la documentation du projet pour les vérifications manuelles de l'API ; aucun compte rendu détaillé ni collection Postman n'est conservé dans le dépôt.

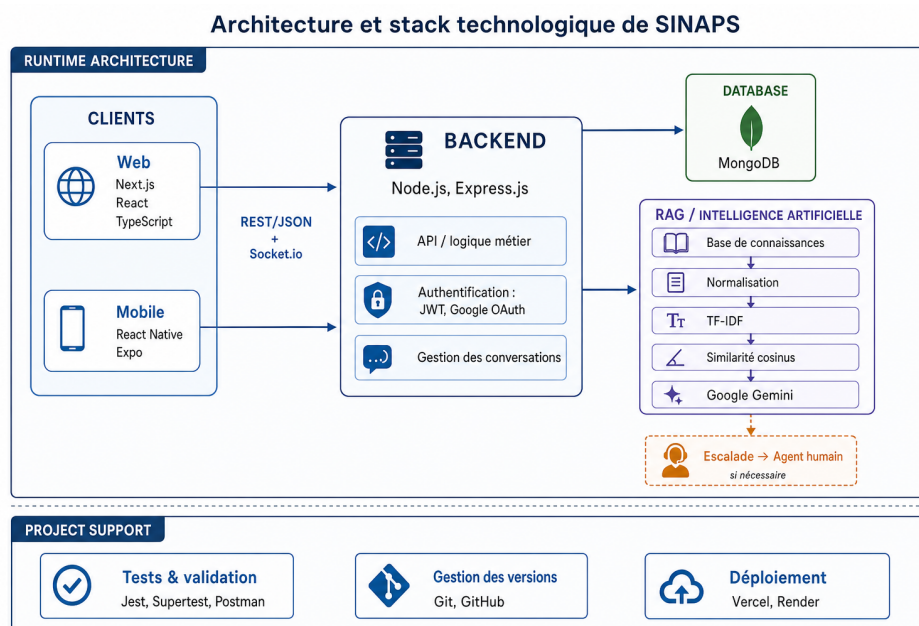


Figure 4.2 – Architecture générale et stack technologique de SINAPS

Justification succincte. Next.js fournit le routage et un frontend React moderne, conformément à l’orientation « React » du sujet. Express est l’API demandée côté Node. MongoDB Atlas évite d’administrer un serveur local de production. Gemini a été retenu (et utilisé effectivement) plutôt qu’OpenAI. Socket.io couvre le besoin de réactivité du chat et de la file agent. JWT sépare clairement sessions client et sessions agent.

4.1.1 Déploiement et hébergement

La chaîne de déploiement vérifiée est **GitHub** → **Vercel (Web)** et **GitHub** → **Render (Backend)**. Le frontend public est accessible à <https://sinaps-pfa.vercel.app> et utilise l’API Render <https://sinaps-backend-jm1h.onrender.com/api>. L’endpoint `GET /api/health` a répondu avec le statut HTTP 200, puis une identification par nom et e-mail, la création d’une conversation et un échange avec réponse IA ont été vérifiés depuis le Web. Cette vérification fonctionnelle a été réalisée par une exécution externe ; la configuration complète Vercel/Render et les secrets ne sont pas versionnés et ne sont donc pas reproductibles depuis le dépôt seul. Elle ne constitue pas une preuve de fonctionnement natif Android ou iOS.

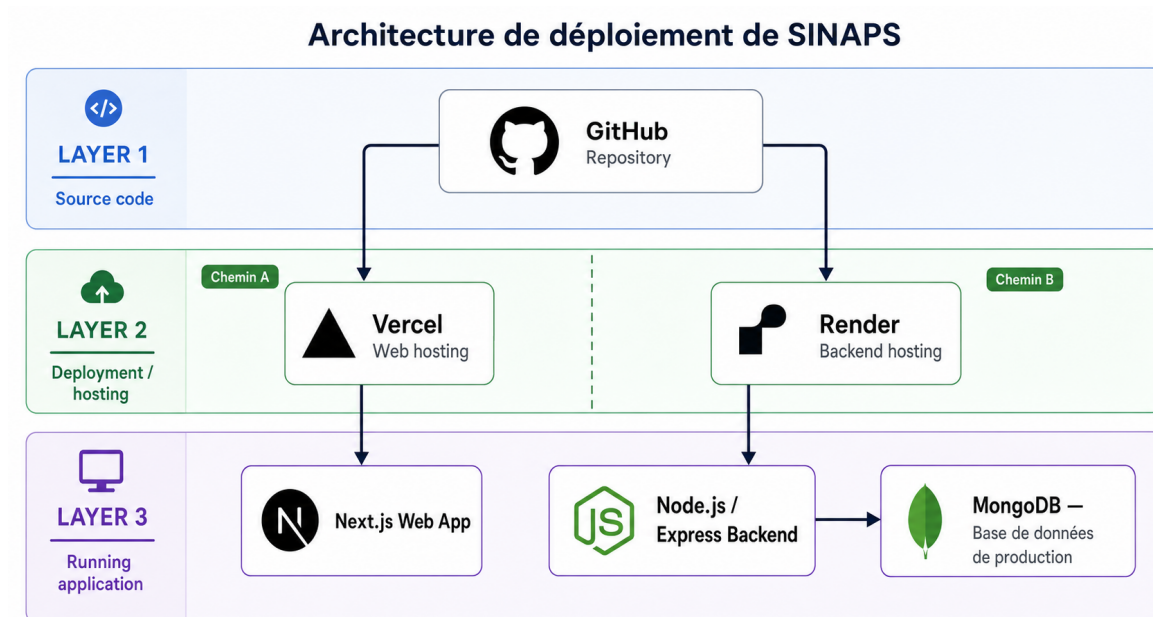


Figure 4.3 – Architecture de déploiement et d’hébergement de SINAPS

Pour la persistance distante documentée, MongoDB Atlas est fourni au backend via `MONGO_URI`. Le code de connexion accepte également une base MongoDB locale ou une base en mémoire pour le développement et les tests ; ces variantes ne doivent pas être confondues avec Atlas. La vérification effectuée dans ce rapport porte sur l’API publique et le parcours Web ; les paramètres privés de la base distante ne sont pas exposés.

Les alternatives du sujet (Vue.js, Flutter, Firebase, Auth0, Kafka, LangChain) **n’ont pas été mises en uvre**.

4.2 Organisation du code

Le dépôt est organisé autour de trois sous-projets : frontend web, backend et client mobile.

Table 4.2 – Arborescence simplifiée

Chemin	Rôle
<code>sinaps-project/app/</code>	Routes : <code>/</code> , <code>/login</code> , <code>/agent</code> , <code>/agent/signup</code> , <code>/admin</code>
<code>sinaps-project/components/chat/</code>	Chat client (fil, en-tête, compositeur, satisfaction, prompts)
<code>sinaps-project/components/admin/</code>	Tableau de bord administrateur
<code>sinaps-project/lib/api.ts</code>	Client HTTP et gestion des jetons
<code>sinaps-project/lib/socket.ts</code>	Client Socket.io
<code>sinaps-backend/server.js</code>	HTTP + Socket.io, CORS, montage des routes
<code>sinaps-backend/models/</code>	Schémas Mongoose
<code>sinaps-backend/controllers/</code>	Logique métier
<code>sinaps-backend/middleware/auth.js</code>	Gardes JWT
<code>sinaps-backend/services/</code>	<code>ragService.js</code> , <code>geminiService.js</code>
<code>sinaps-backend/data/knowledgeBase.js</code>	FAQ d'exemple
<code>sinaps-backend/tests/</code>	Suites Jest
<code>sinaps-backend/seed.js</code>	Jeu de démonstration
<code>sinaps-mobile/src/</code>	Client Expo / React Native : authentification, chat, temps réel, écrans agent et administrateur

Le frontend écoute par défaut `NEXT_PUBLIC_API_URL=http://localhost:5000/api`. Le backend écoute le port **5000**. Un endpoint `GET /api/health` permet de vérifier que l'API est joignable. Aucun manifeste spécifique à Vercel ou Render n'est présent dans le dépôt ; leurs paramètres de déploiement sont documentés dans le README et reposent sur les variables d'environnement indiquées.

4.3 Backend : API REST, middleware et WebSockets

4.3.1 Routes

Table 4.3 – Principales routes HTTP

Méthode	Chemin	Rôle
GET	/api/health	Santé du serveur
POST	/api/users/find-or-create	Session client (Google ou e-mail)
POST	/api/conversations/find-or-create	Conversation active du client
GET	/api/conversations	Liste (agent/admin), filtre <code>status</code> , <code>search</code>
GET	/api/conversations/:id	Fil + messages (propriétaire ou staff)
PATCH	/api/conversations/:id/escalate	Vers humain
PATCH	/api/conversations/:id/de-escalate	Retour IA
PATCH	/api/conversations/:id/assign	Assignment d'agent
PATCH	/api/conversations/:id/close	Clôture ± satisfaction
POST	/api/messages	Envoi ; déclenche l'IA si besoin
POST	/api/conversations/:id/quick-reply	Exécution d'une action rapide
POST	/api/upload	Fichier (session requise)
POST	/api/agents/signup	Inscription <code>pending</code>
POST	/api/agents/login	JWT staff
GET	/api/agents	Liste (admin)
PATCH	/api/agents/:id/approve	Validation
DELETE	/api/agents/:id	Rejet (suppression)
GET	/api/stats	Indicateurs admin

4.3.2 Temps réel

Lors d'une connexion Socket.io, le client peut `join_conversation` ou `leave_conversation`. Le backend émet `message_received` et `conversation_updated` vers la salle. Les événements globaux `conversation_created` et `conversation_updated` servent au rafraîchissement des listes.

4.3.3 Statistiques

`getStats` calcule le nombre total de conversations, les résolutions par IA ou par un humain, la moyenne des notes renseignées et un **temps de réponse moyen** à partir des messages.

4.3.4 Télversement

Multer enregistre les fichiers avec un nom unique. Types acceptés : images, vidéos, PDF et Word. La taille maximale est configurable (`MAX_UPLOAD_SIZE_MB`, défaut 15 Mo) et le type MIME déclaré est filtré.

4.4 Moteur IA

Le fichier `geminiService.js` enchaîne : retrieval → construction du prompt → essai de modèles Gemini → repli RAG textuel. La FAQ d'exemple couvre notamment : suivi de commande, retard, remboursement, mot de passe, code promo. Elle n'est pas un corpus métier confidentiel de SINAPS TEC.

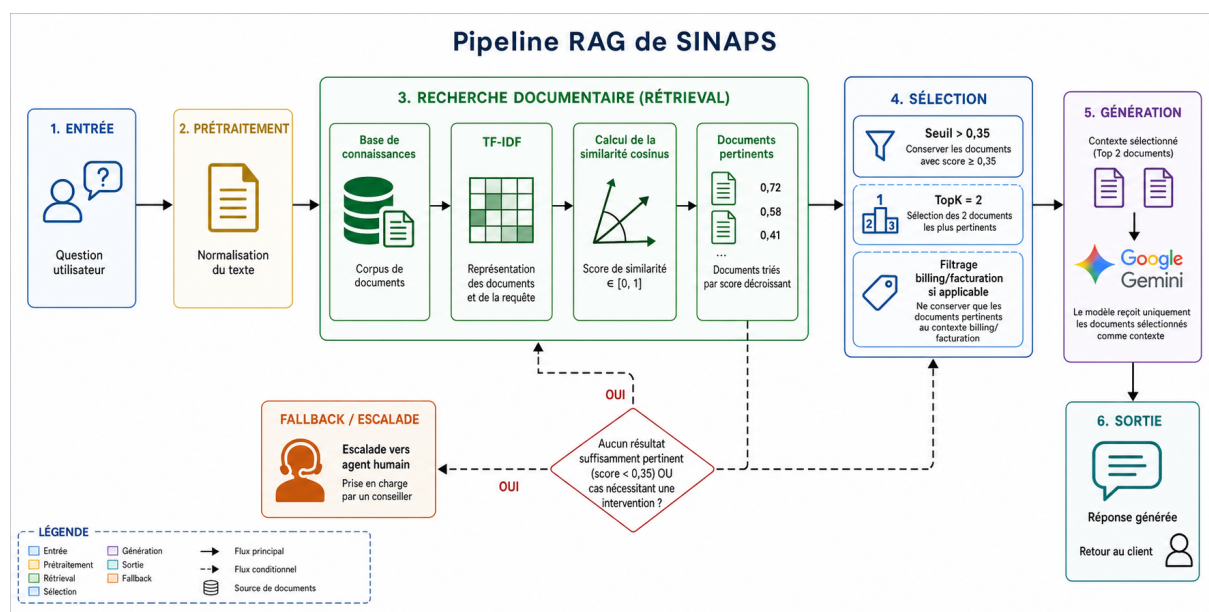


Figure 4.4 – Pipeline de recherche augmentée par génération (RAG) de SINAPS

Le retriever normalise le français (minuscules, accents, apostrophes et ponctuation), tokenise les textes, retire des mots vides et applique une normalisation de variations lexicales. Il calcule ensuite des vecteurs TF-IDF et plusieurs similarités cosinus : question (poids

0,5), mots-clés (0,3), texte combiné (0,1) et réponse (0,05), avec un bonus pour les mots-clés pertinents. Seuls les documents dont le score est strictement supérieur à **0,35** sont retenus, avec un maximum de **deux documents** ($\text{topK} = 2$). Les demandes de facturation sont limitées aux documents de catégorie **billing**. Sans clé Gemini, en cas d'échec du modèle ou sans document pertinent, le service utilise la meilleure réponse locale ou propose une mise en relation humaine.

4.5 Frontend

4.5.1 Espace client

Après identification (`ClientEntryForm`, bouton Google et/ou formulaire), `SupportChatApp` charge le fil, affiche `ChatThread`, `QuickPrompts`, `MessageComposer`, et les actions d'escalade / désescalade dans `ChatHeader`. Un dialogue de satisfaction s'ouvre à la clôture.

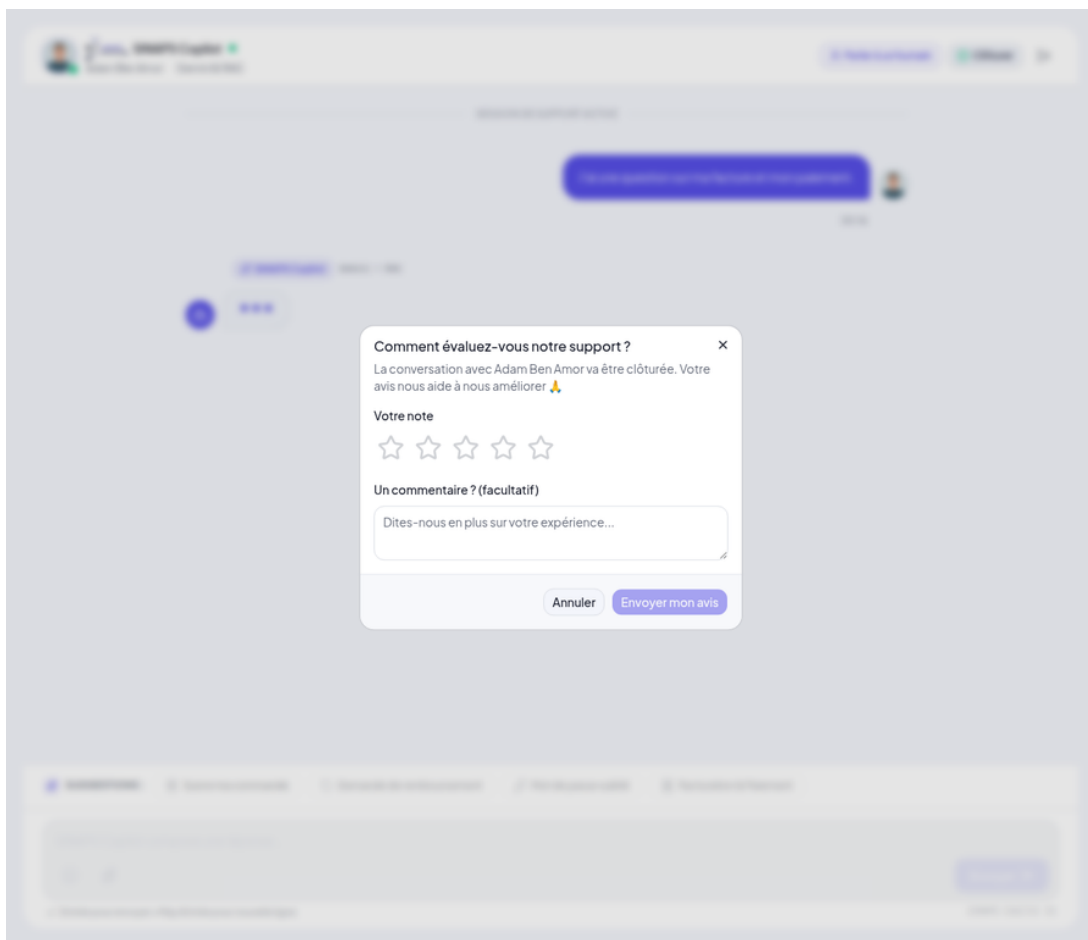


Figure 4.5 – Évaluation de la satisfaction à la clôture d'une conversation

Les réponses IA peuvent porter des **actions rapides** : confirmer que la réponse a suffi, poser une autre question ou demander un agent humain. Une action est envoyée comme message client et son état de chargement est affiché ; les boutons ne sont proposés que sur le dernier message IA éligible. Le compositeur est verrouillé pendant la génération IA afin d'éviter les doublons.

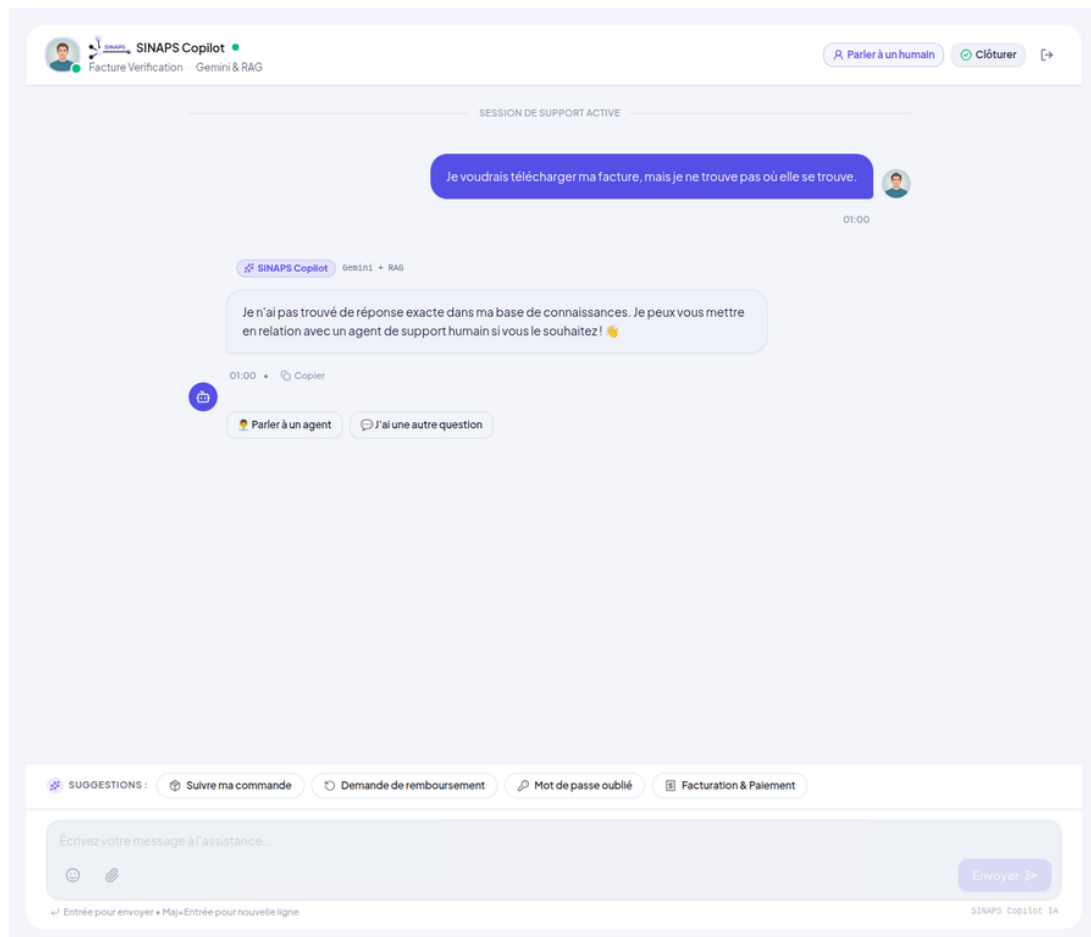


Figure 4.6 – Conversation client avec réponse RAG/Gemini et actions rapides

4.5.2 Espace agent

La page `/agent` est protégée (AuthGuard). L'agent se connecte sur `/login`. L'inscription se fait sur `/agent/signup` (compétences en texte). Tant que le compte est **pending**, le login renvoie une erreur 403.

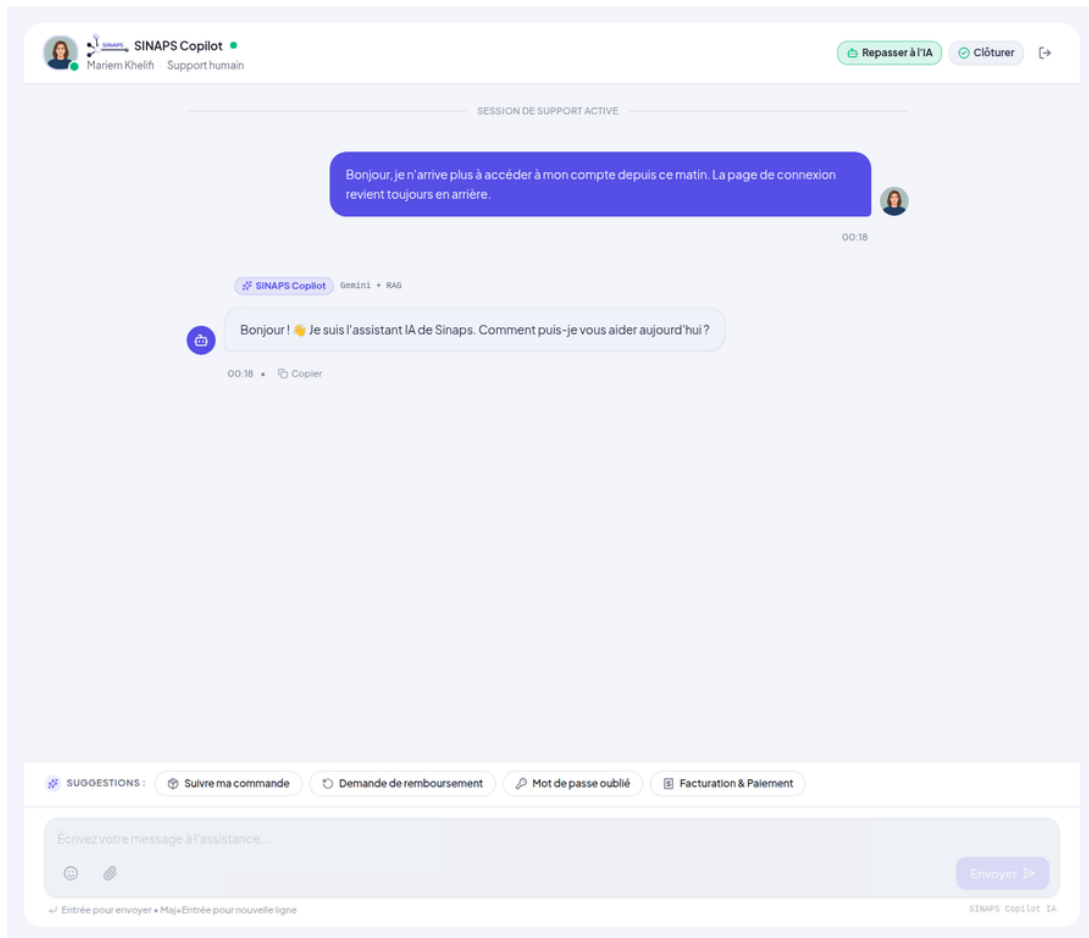


Figure 4.7 – Escalade d’une conversation vers la file des agents

4.5.3 Espace administrateur

Le tableau de bord affiche les indicateurs opérationnels de satisfaction et de délai de réponse, la liste des agents à valider (compétences en badges), et un historique avec recherche (nom / e-mail client) et onglets de statut. Les conversations marquées comme données de test sont exclues des indicateurs et des listes opérationnelles par défaut. Les dernières corrections ont également stabilisé le chargement concurrent des conversations, le rendu des horodatages et les avatars de repli côté client.

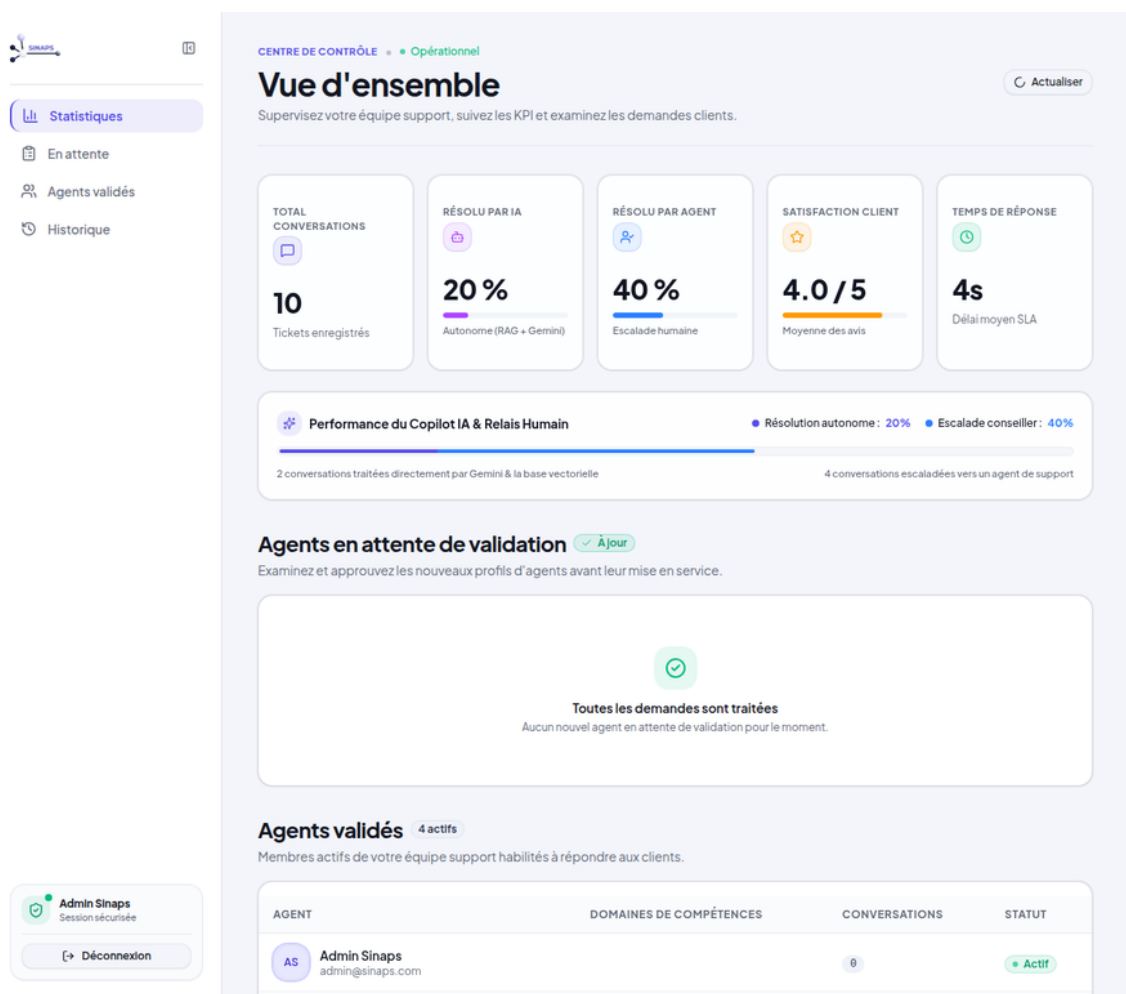


Figure 4.8 – Tableau de bord administrateur de SINAPS TEC

4.5.4 Client mobile

Le dossier `sinaps-mobile/` contient un client Expo / React Native connecté aux mêmes routes HTTP et au même serveur Socket.io que le client web. Il propose l'authentification client par nom / e-mail ou Google, l'authentification agent / administrateur par e-mail et mot de passe, la persistance de session, le chat, les pièces jointes, l'indicateur de frappe, l'escalade, le retour vers l'IA, la clôture avec satisfaction et le mode sombre. Des écrans distincts permettent également à un agent de consulter la file, de prendre en charge une conversation, de répondre et de la résoudre, ainsi qu'à un administrateur de consulter les statistiques, les agents et l'historique.

Le mobile n'est pas visuellement identique au web, car son interface est adaptée aux petits écrans. Il reprend toutefois des actions rapides **contextuelles** attachées à certaines

réponses IA, l'escalade vers un agent humain, le retour vers l'IA, la clôture avec satisfaction et l'indicateur d'état Socket.io. Son compositeur propose les images et documents; le parcours vidéo exposé par le web n'a pas été ajouté. L'inscription d'un agent reste disponible uniquement sur `/agent/signup`. La connexion Google mobile dépend en outre d'une build native Expo configurée. Enfin, les valeurs de secours prévues dans l'écran administrateur mobile lorsque les statistiques sont absentes ne doivent pas être considérées comme des mesures de production.

Le client mobile a été lancé en Expo Web sur `http://localhost:8081` avec l'API Render. Le parcours vérifié comprend l'identification directe, l'envoi d'un message, une réponse IA fondée sur le RAG et les actions rapides contextuelles, ainsi que l'indicateur Socket.io « En direct ». Une escalade explicite vers un agent humain fait apparaître les états « En attente » et « Agent demandé »; le contrôle « Mode IA » permet ensuite le retour vers l'assistant, avec une nouvelle réponse IA et des actions rapides. La clôture déclenche enfin le dialogue d'évaluation à cinq étoiles. Cette vérification ne couvre ni une build native Android/iOS ni Google Sign-In natif. Le dépôt contient la configuration TypeScript du client mobile, mais aucune suite de tests automatisés mobile n'est présente.

L'écran d'identification directe par nom et e-mail est présenté en figure 4.9. La figure 4.10 montre ensuite une conversation mobile avec une réponse de l'assistant IA fondée sur le RAG et les actions rapides disponibles pour le client.

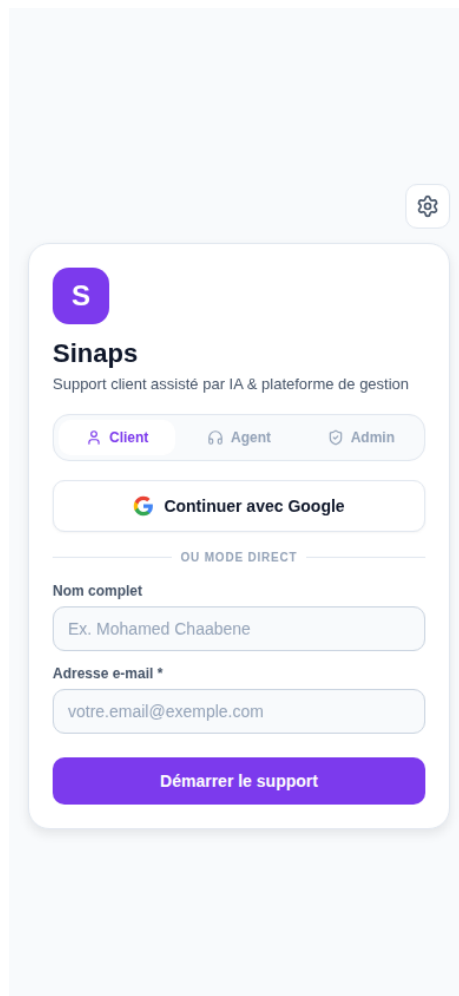


Figure 4.9 – Écran d'identification directe du client dans l'application mobile SINAPS

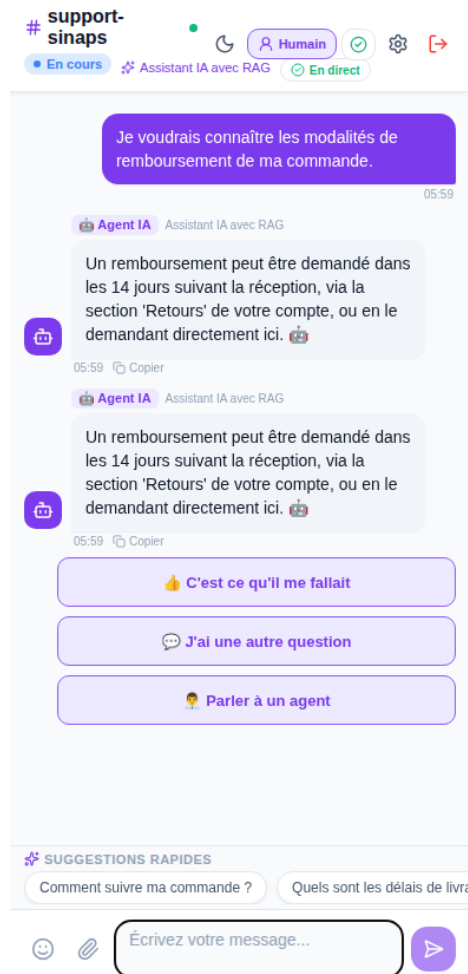


Figure 4.10 – Conversation mobile avec réponse de l’assistant IA fondée sur le RAG et actions rapides

La figure 4.11 illustre la demande explicite de prise en charge humaine, les états « En attente » et « Agent demandé », ainsi que le contrôle de retour « Mode IA ». Après la résolution, la figure 4.12 présente le dialogue de notation et de clôture de la conversation.

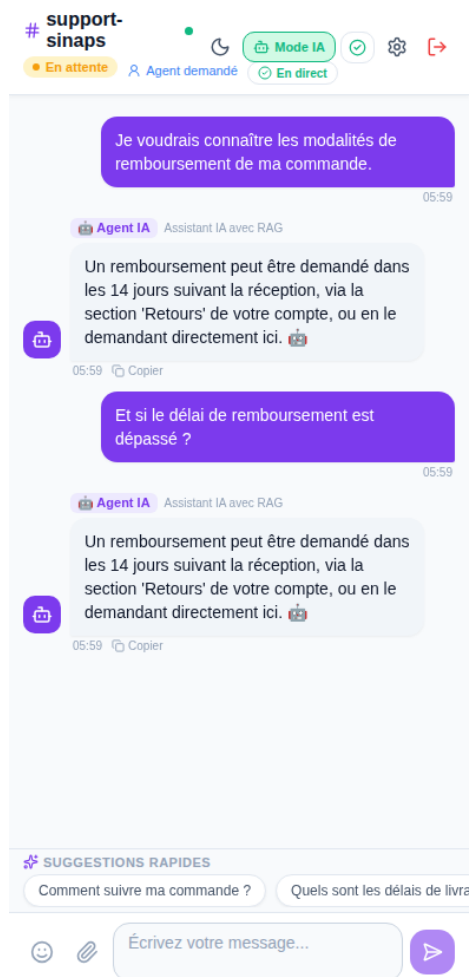


Figure 4.11 – Escalade explicite vers un agent humain : états En attente, Agent demandé et retour Mode IA

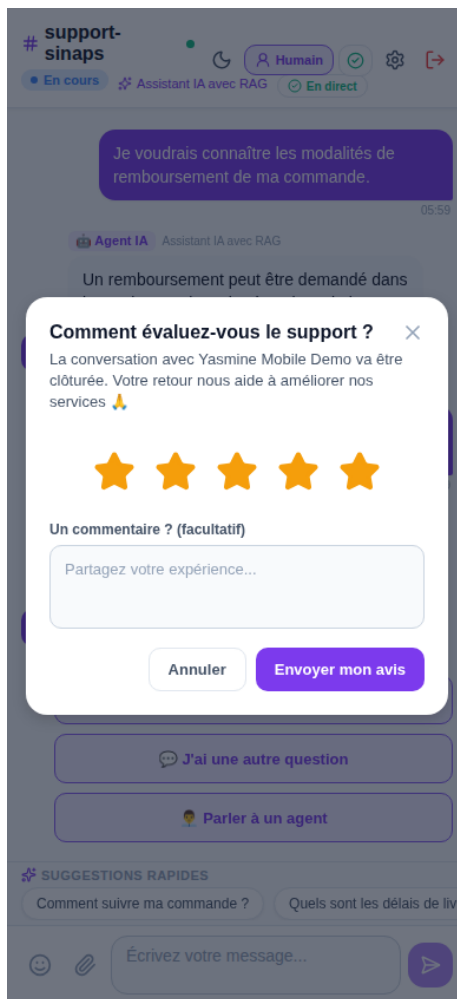


Figure 4.12 – Évaluation de la satisfaction et clôture d’une conversation mobile

4.6 Données de démonstration

Le script `npm run seed` recrée des comptes de démonstration avec un mot de passe défini par le script de seed, ainsi que des utilisateurs et conversations de démonstration. Ces comptes servent aux captures et aux tests manuels ; ils constituent uniquement des données locales ou de test et non une population réelle de production.

4.7 Mise en route (développement local)

```
# MongoDB Atlas : renseigner MONGO_URI
cd sinaps-backend && npm install && npm run seed && npm run dev
cd sinaps-project && npm install && npm run dev
```

Listing 4.1 – Démarrage local (d'après le README du projet)

Variables nécessaires côté backend : `PORT`, `MONGO_URI`, `JWT_SECRET`, `GEMINI_API_KEY`, `GOOGLE_CLIENT_ID`; côté frontend : `NEXT_PUBLIC_API_URL`, `NEXT_PUBLIC_GOOGLE_CLIENT_ID`. Sans clés Google/Gemini, des modes dégradés existent (identification directe, réponses RAG locales). Le dépôt confirme l'intégration de Google Sign-In, Gemini et MongoDB Atlas, mais ne fournit pas de compte rendu détaillé d'une recette manuelle en conditions réelles.

Chapitre 5

Tests et validation

5.1 Introduction

Cette phase vise à vérifier que les principaux comportements du backend de SINAPS correspondent aux besoins retenus : authentification, conversations, messages, traitement IA, prise en charge humaine, administration et téléversement. Les résultats présentés sont fondés sur les tests présents dans le dépôt et sur l'exécution de la suite backend. Aucun pourcentage de couverture, aucun temps de réponse mesuré en production et aucun recettage utilisateur formalisé n'est avancé.

5.2 Stratégie de test

La stratégie combine plusieurs niveaux de vérification réellement présents dans le projet :

- des tests automatisés backend, exécutés avec Jest ;
- des tests d'intégration HTTP avec Supertest, utilisant MongoDB Memory Server pour isoler les données ;
- des tests unitaires ciblés, notamment pour le middleware d'authentification, le formatage des horodatages et le retriever RAG ;
- des tests du téléversement avec Multer, des actions rapides, du cycle de résolution et du filtrage des données de démonstration ;
- Postman est mentionné dans la documentation du projet pour les vérifications manuelles de l'API ; aucun compte rendu détaillé ni collection Postman n'est conservé

dans le dépôt.

Le frontend Next.js et l'application mobile ne contiennent pas de suite de tests automatisés dédiée dans le dépôt. Une validation manuelle des interfaces est possible, mais ses dates, scénarios détaillés et résultats ne sont pas documentés de manière vérifiable dans les sources disponibles.

La suite backend est lancée avec la commande suivante :

```
cd sinaps-backend  
npm test
```

Listing 5.1 – Exécution de la suite backend

5.3 Tests fonctionnels

Les scénarios fonctionnels ci-dessous sont vérifiés principalement à travers les tests d'API et d'intégration. Ils correspondent aux flux métier implémentés, sans présenter ces vérifications comme une recette complète des interfaces Web ou Mobile.

Table 5.1 – Scénarios fonctionnels vérifiés par les tests backend

Scénario	Vérification principale	Résultat
Authentification et autorisation	Jetons client, agent et administrateur ; rôles incompatibles et accès aux conversations	Réussi
Création et consultation d’une conversation	Conversation active, propriétaire autorisé, autre client refusé	Réussi
Réponse IA / RAG	Message client, réponse IA lorsque <code>handledBy = ia</code> , recherche locale et repli sans clé Gemini	Réussi
Escalade et prise en charge humaine	Passage à <code>humain/en_attente</code> , assignation, réponse agent et prévention d’un changement non autorisé	Réussi
Retour vers l’IA et actions rapides	Désescalade, confirmation, nouvelle question et demande d’agent	Réussi
Clôture et satisfaction	Passage à <code>resolu</code> , note entière de 1 à 5 et refus d’une note invalide	Réussi
Administration	Validation d’agents, statistiques et exclusion des données <code>isTestData</code> par défaut	Réussi
Pièces jointes	Session obligatoire, types autorisés et limite de taille	Réussi

5.4 Tests backend automatisés

5.4.1 Intégration API

Le fichier `api.test.js` vérifie notamment :

Table 5.2 – Exemples de cas d’intégration API

Cas	Attendu
Création d’utilisateur et jeton client	Code 200 avec un <code>token</code>
<code>find-or-create</code> sans session	Code 401
Création d’une conversation authentifiée	<code>handledBy = ia</code>
Lecture sans session ou par un autre client	Codes 401 ou 403
Lecture par le propriétaire	Code 200
Escalade et désescalade	Modes et statuts mis à jour
Clôture avec note 5 ou note 0	Résolution ; seule la note valide est enregistrée
Message humain ou client selon le jeton	Accepté avec le rôle approprié, refusé sinon
Injection d’un message <code>sender = ia</code>	Code 403

5.4.2 Authentification, conversations et messages

`auth-middleware.test.js` couvre l’absence de header, les jetons invalides, les rôles inadaptés et l’appartenance à une conversation pour `requireClientAuth`, `requireConversationAccess` et `requireSenderAuth`. Les tests du flux agent vérifient l’escalade, l’assignation, la réponse humaine, la résolution, la protection contre la prise d’une conversation déjà attribuée et le scénario complet IA → humain → résolution.

Les tests d’actions rapides vérifient notamment leur stockage, leurs identifiants, leur idempotence, la mise à jour de `lastActivityAt` et les actions de confirmation, de nouvelle question et d’escalade. Les tests d’horodatage contrôlent la présence et la sérialisation de `createdAt` lors du chargement d’une conversation agent.

5.4.3 Upload et retriever RAG

`upload.test.js` vérifie le refus d’une requête sans session, le refus d’un type interdit, l’acceptation d’un fichier JPEG côté client et d’un PDF côté agent, la limite de taille fixée par le test à 1 Mo et la sécurisation de l’extension d’un sous-type image atypique.

`rag.test.js` contrôle que des formulations liées au suivi de commande ou au remboursement récupèrent l’entrée correspondante, qu’une requête sans rapport ne retourne aucun document et qu’une question de facturation n’est pas dirigée vers une réponse de suivi de commande. Le repli de `getAIResponse` sans clé Gemini est également vérifié. Ces tests

valident le retriever local et le repli ; ils ne mesurent pas la qualité linguistique de Gemini en production.

5.4.4 Résultat de la suite

La dernière exécution locale de la commande `npm test` a produit le résultat suivant :

13 suites passées, 205 tests passés, 0 échec, 0 snapshot

Cette exécution couvre le backend. Elle ne constitue ni une mesure de couverture ni une garantie de performance en production.

5.5 Résultats et anomalies corrigées

Les tests dédiés documentent plusieurs problèmes rencontrés puis corrigés pendant le développement :

- **État de prise en charge IA** : les conversations nouvellement créées devaient conserver `handledBy = ia` afin de déclencher correctement la réponse IA. Les tests couvrent aussi le comportement par défaut des conversations héritées sans valeur explicite.
- **Persistance et chargement des messages** : le traitement attend la création du message IA avant de répondre à la requête HTTP, et le rechargement d'une conversation retourne les messages persistés avec leurs horodatages.
- **Réponses de facturation** : une question concernant une facture ne doit pas recevoir par erreur la réponse relative au suivi d'une commande ; le retriever et le repli sont testés sur ce cas.
- **Authentification des messages humains** : un message `sender = humain` exige une session agent ou administrateur, tandis qu'un client ne peut envoyer qu'un message dans sa propre conversation.
- **Escalade et assignation** : les tests vérifient la séparation entre prise en charge IA et humaine, l'assignation à un agent et la prévention d'un changement silencieux d'agent.
- **Données de démonstration** : les conversations marquées `isTestData` sont exclues des listes administratives par défaut afin de ne pas fausser les indicateurs opérationnels.

5.6 Validation finale

La version évaluée dispose d'une suite backend reproductible qui couvre les principaux flux d'API, la persistance MongoDB en mémoire, l'authentification, les conversations, le RAG local, les réponses de repli, les pièces jointes, les actions rapides, les horodatages et l'administration. L'exécution confirmée aboutit à 205 tests réussis répartis sur 13 suites.

Les limites restantes sont les suivantes :

- aucun test E2E de navigateur avec Playwright ou Cypress n'est présent dans le dépôt ;
- aucun test de charge Socket.io n'est présent ;
- aucune recette formelle signée par l'encadrant n'est documentée ;
- les indicateurs du tableau de bord dépendent des données effectivement présentes ; le jeu de démonstration est marqué `isTestData` et exclu par défaut ;
- aucune preuve de déploiement public ni mesure de production n'est fournie dans le dépôt ;
- les tests automatisés ne permettent pas de conclure à une validation complète des interfaces Web et Mobile.

5.7 Conclusion

Les tests backend automatisés ont permis de vérifier les principaux comportements fonctionnels et plusieurs corrections importantes de SINAPS. Les résultats obtenus sont reproductibles et couvrent les flux essentiels de l'API, tout en laissant clairement identifiées les limites liées aux tests d'interface, de charge et de validation en production.

Conclusion générale

Ce PFA a consisté à concevoir et réaliser **Op Support IA**, une solution web et mobile de support client dans laquelle un agent IA s'appuie sur une petite base de connaissances, un opérateur humain peut prendre le relais, et un administrateur pilote comptes, historique et indicateurs.

Le travail a été mené **seul**, à **distance**, de façon **informelle**, du 22 juin au 6 septembre 2026, sur un sujet proposé par **Kais Gharred** (SINAPS TEC). L'application livrée comprend un frontend Next.js, un backend Express, une persistance MongoDB, une authentification Google et JWT, un moteur RAG lexical couplé à Gemini, des Web-Sockets, des actions rapides, un téléversement de fichiers contrôlé, des horodatages et une suite de tests backend. MongoDB Atlas est le service distant documenté pour la persistance ; ses paramètres privés ne sont pas détaillés dans ce rapport.

Par rapport au document de sujet, le cur fonctionnel web et mobile (chat, RAG, escalade, agents, admin, contenus riches sous forme de pièces jointes, satisfaction) est en place. Le mobile n'est toutefois pas fonctionnellement identique au web : il propose des actions rapides contextuelles attachées aux réponses IA éligibles, avec une présentation adaptée aux petits écrans, l'inscription agent reste web, et la connexion Google nécessite une build native configurée. Le **routage par compétences** et le **RAG vectoriel** n'ont pas été réalisés — par choix de périmètre, compte tenu de la durée et du caractère individuel du stage. Le déploiement Web public et l'API Render ont été vérifiés fonctionnellement par une exécution externe ; la configuration complète Vercel/Render et les secrets ne sont pas versionnés et ne sont donc pas reproductibles depuis le dépôt seul.

La version finale a été vérifiée par **13 suites et 205 tests backend passés**. Les corrections finales ont notamment porté sur les réponses IA hors sujet, les indicateurs d'escalade, le chargement concurrent des conversations, la sérialisation des horodatages, les avatars

de repli, les actions rapides et l'exclusion des données de démonstration des indicateurs. En complément, l'API publique Render et le parcours Web de production ont été vérifiés, ainsi que le parcours client Expo Web jusqu'à la réponse IA et la connexion Socket.io. Ces vérifications ne remplacent pas une recette native Android/iOS.

Sur le plan pédagogique, la principale difficulté a été l'**apprentissage simultané** de l'écosystème utilisé : Node.js, Express, React, Next.js, et les notions de RAG. Cette montée en compétence a été nécessaire pour livrer une chaîne fullstack cohérente plutôt qu'un prototype d'interface isolé.

Les perspectives naturelles, si le projet se poursuivait (par exemple dans un PFE), seraient : déployer l'application, remplacer le retriever TF-IDF par des embeddings si la base s'enrichit, affecter les files selon les compétences, durcir l'upload (inspection des octets), et ajouter des tests de bout en bout sur l'UI. Ces pistes ne sont pas des engagements déjà tenus.

Nous remercions à nouveau l'encadrant professionnel et l'ensemble du corps enseignant de l'ISIMS pour l'accompagnement et les connaissances mobilisées dans ce premier projet de fin d'année d'ingénierie logicielle.

Bibliographie

- [1] Express.js, *Fast, unopinionated, minimalist web framework for Node.js*. Documentation officielle. <https://expressjs.com/> (consulté en 2026).
- [2] Vercel, *Next.js Documentation*. <https://nextjs.org/docs> (consulté en 2026).
- [3] Mongoose, *Elegant MongoDB object modeling for Node.js*. <https://mongoosejs.com/> (consulté en 2026).
- [4] MongoDB, *MongoDB Atlas Documentation*. <https://www.mongodb.com/docs/atlas/> (consulté en 2026).
- [5] Socket.IO, *Documentation*. <https://socket.io/docs/> (consulté en 2026).
- [6] Auth0, *JSON Web Tokens*. <https://jwt.io/> (consulté en 2026).
- [7] Google AI, *Gemini API*. <https://ai.google.dev/> (consulté en 2026).
- [8] Google Identity, *Authenticate with a backend server*. <https://developers.google.com/identity/gsi/web> (consulté en 2026).
- [9] Jest, *Delightful JavaScript Testing*. <https://jestjs.io/> (consulté en 2026).
- [10] Lewis, P. et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS, 2020. (référence conceptuelle du terme RAG ; l'implémentation du PFA est un retriever TF-IDF local, distinct des modèles de l'article.)
- [11] GHARRED, Kais (encadrant SINAPS TEC), *Projet Stage – Op Support IA*. Document de cadrage interne, 2026.
- [12] Chaabene, M., *Documentation technique et utilisateur – Projet Op Support IA (Sinaps)*, fichier `sinaps-project/README.md` du dépôt, 2026.

Annexe A

Annexes

A.1 Glossaire des statuts de conversation

Table A.1 – Statuts et modes de prise en charge

Champ	Valeur	Signification opérationnelle
status	en_cours	Conversation active (IA ou humain déjà en charge)
status	en_attente	Escaladée, en file côté agents
status	resolu	Clôturée
handledBy	ia	Prochaine réponse automatique possible
handledBy	humain	Réponses attendues d'un opérateur

A.2 Exemple d'entrée de la base de connaissances

Les extraits suivants sont des **exemples** utilisés par le retriever, et non une documentation officielle de produits SINAPS TEC.

```
{
  question: "Comment suivre ma commande ?",
  answer: "Vous pouvez suivre votre commande depuis votre espace
          client, section 'Mes commandes'. Un numéro de suivi
          est envoyé par email dès l'expédition."
}
```

Listing A.1 – Extrait de `knowledgeBase.js`

A.3 Comptes du script de seed (environnement de démo)

Ces identifiants figurent dans le README du projet à des fins de démonstration locale. Ils ne doivent pas être réutilisés en production.

Table A.2 – Comptes créés par `npm run seed`

Rôle	E-mail	Mot de passe (seed)	Accès
Admin	admin@sinaps.com	password123	/admin
Agent	sarah.benali@sinaps.com	password123	/agent
Agent	karim.mansouri@sinaps.com	password123	/agent
Pending	youssef.mehdi@sinaps.com	password123	login refusé tant que non validé